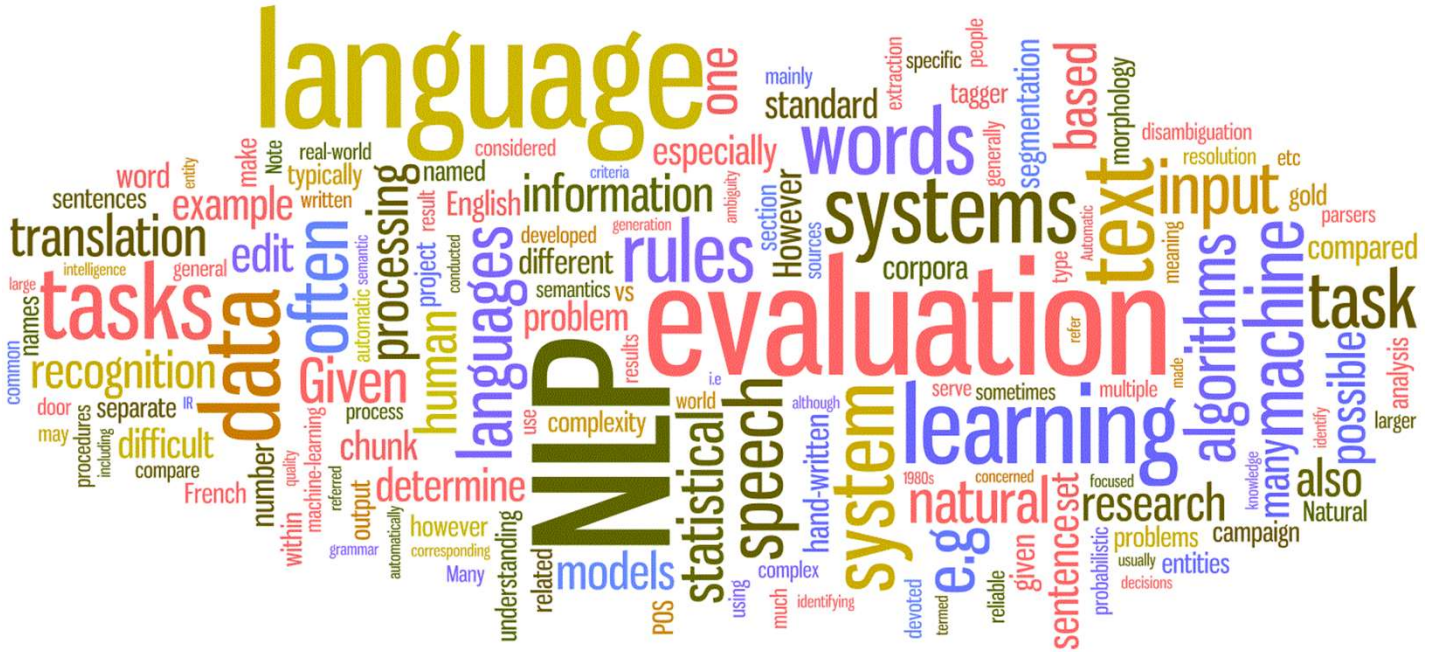


Lecture 10: Text Catergorization

Xu Ruifeng

Harbin Institute of Technology, Shenzhen

Last Time

- Dependency Syntax
- Dependency Parsing
- Neural-based Dependency Parsing
- Parsing Evaluation
- Parsing Resources

Today's Class

- Text Categorization: Background
- Document Representation
- Feature Selection
- Feature Generation
- Text Categorization Algorithms
  - Rocchio
  - K-Nearest Neighbour
  - SVM
  - Deep learning-based methods
  - Pretraining methods
  - Other methods
- Text Categorization Evaluation

Introduction

- Purpose: classification of natural language texts into a set of predefined labels.

- Spam filtering
- News text classification
- Emotion classification
- Subject classification
- ……

Is this spam?

Subject: **Important notice!**
From: Stanford University <newsforum@stanford.edu>
Date: October 28, 2011 12:34:16 PM PDT
To: undisclosed-recipients:;

Greats News!

You can now access the latest news by using the link below to login to Stanford University News Forum.

http://www.123contactform.com/contact-form-StanfordNew1-236335.html

Click on the above link to login for more information about this new exciting forum. You can also copy the above link to your browser bar and login for more information
about the new services.

Positive or Negative movie review?

\+ *...zany characters and **richly** applied satire, and some **great** plot twists*

– *It was **pathetic**. The **worst** part about it was the boxing scenes...*

\+ *...**awesome** caramel sauce and sweet toasty almonds. I **love** this place!*

– *...**awful** pizza and **ridiculously** overpriced...*

What is the label of this News?

澎湃 政务：三亚发布 2021-11-23 15:45

记者从省市场监管局了解到，今年以来，该局将学校食堂“互联网+明厨亮灶”全覆盖列入“我为群众办实事”任务清单，持续开展校园食品安全守护行动，目前已成功实现全省3358家持证学校食堂“互联网+明厨亮灶”全覆盖。

该局与教育部门密切协作，联合部署推进此项工作。相关行动将学校食堂的视频监控信号通过互联网接入“智慧监管”平台，师生、家长、监管人员可随时查看资质信息、食品来源、人员健康证等，并在线观看食品制作过程，将“闲人免进”的后厨暴露在阳光之下，实现厨房透明化、操作可视化、监管实时化，有效压实了学校食品安全主体责任。

校园食品安全守护行动中，省市场监管局针对部分学校食堂食品安全制度陈旧过时、可操作性差等问题，联合教育部门编制了学校食品安全管理体系，对学校食堂管理中关键岗位职责、食品安全制度、具体操作流程等方面进行了规范和明确。同时，对全省学校食堂、学校集体用餐配送单位及校园周边食品经营户实施全覆盖监督检查，共排查出1180家次单位存在风险隐患，下达责令整改通知书586份。

下一步，海南还将强化风险隐患排查，推进学校食堂标准化建设，并在巩固现有“互联网+明厨亮灶”覆盖率的基础上，提高智慧监管平台公众参与度，着力构建行业规范自律、群众参与监督、政府高效监管的“三位一体”共治格局。

来源：海南日报客户端

?

- Military
- Politics
- Internet
- Entertainment
- ……

Text Categorization

- Purpose: classification of natural language texts into a set of predefined labels.
- Given:
  - A description of an instance, $x \in X$, where X is the *instance language* or *instance space*.
  - A fixed set of categories:

    $C=\{c_1, c_2,\ldots c_n\}$
- Determine:
  - The category of x: $c(x) \in C$, where $c(x)$ is a categorization function whose domain is X and whose range is C.

Examples of Text Categorization

- LABELS=BINARY
  - “spam” / “not spam”
- LABELS=TOPICS
  - “finance” / “sports” / “asia”
- LABELS=OPINION
  - “positive” / “negative” / “neutral”
- LABELS=AUTHOR
  - “Shakespeare” / “Marlowe” / “Ben Jonson”
  - The Federalist papers

Classification types

- Document Membership
  - Binary Classification
  - Multi-class Classification
  - Multi-labels Classification

- Hard vs Ranking Classifiers
  - Hard = Decisive!
  - Ranking = Probabilistic

Supervised Learning

- Training classifier based on set of labeled documents
- Training set vs. Test set

Input:

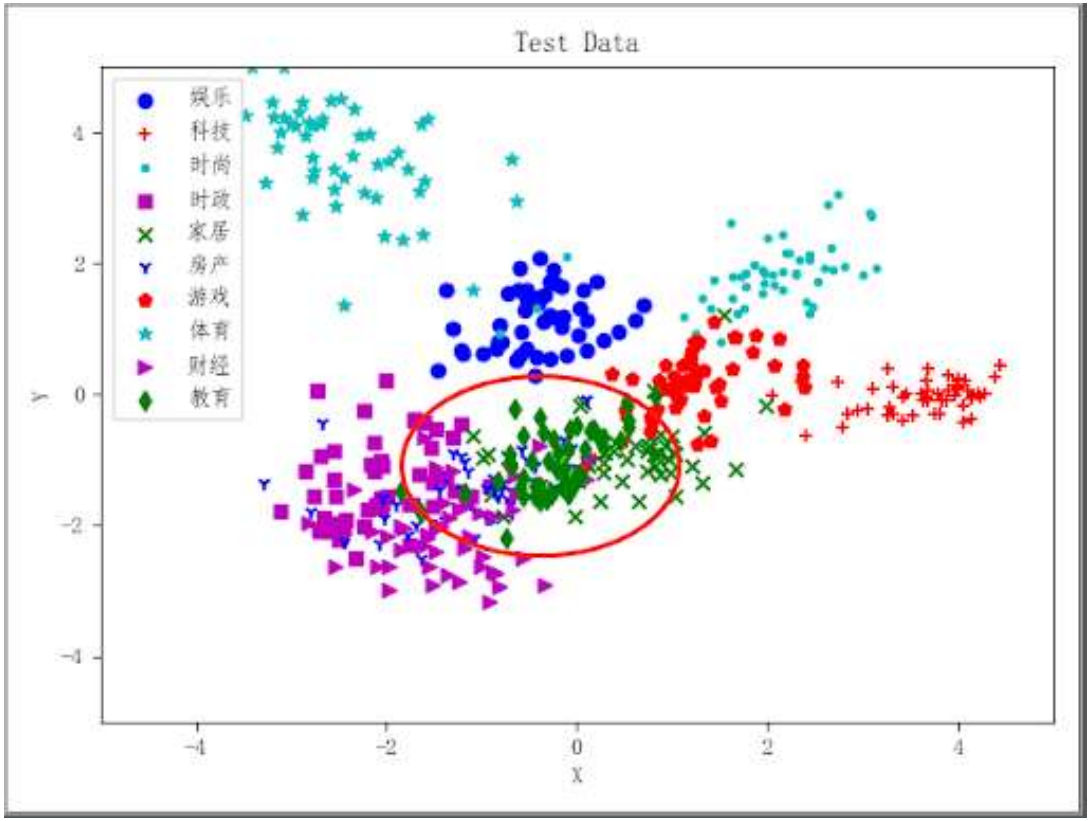
- a document d
- a fixed set of classes $C = \{c_1, c_2, \ldots, c_J\}$
- A training set of m hand-labeled documents $(d_1,c_1),\ldots,(d_m,c_m)$

Output:

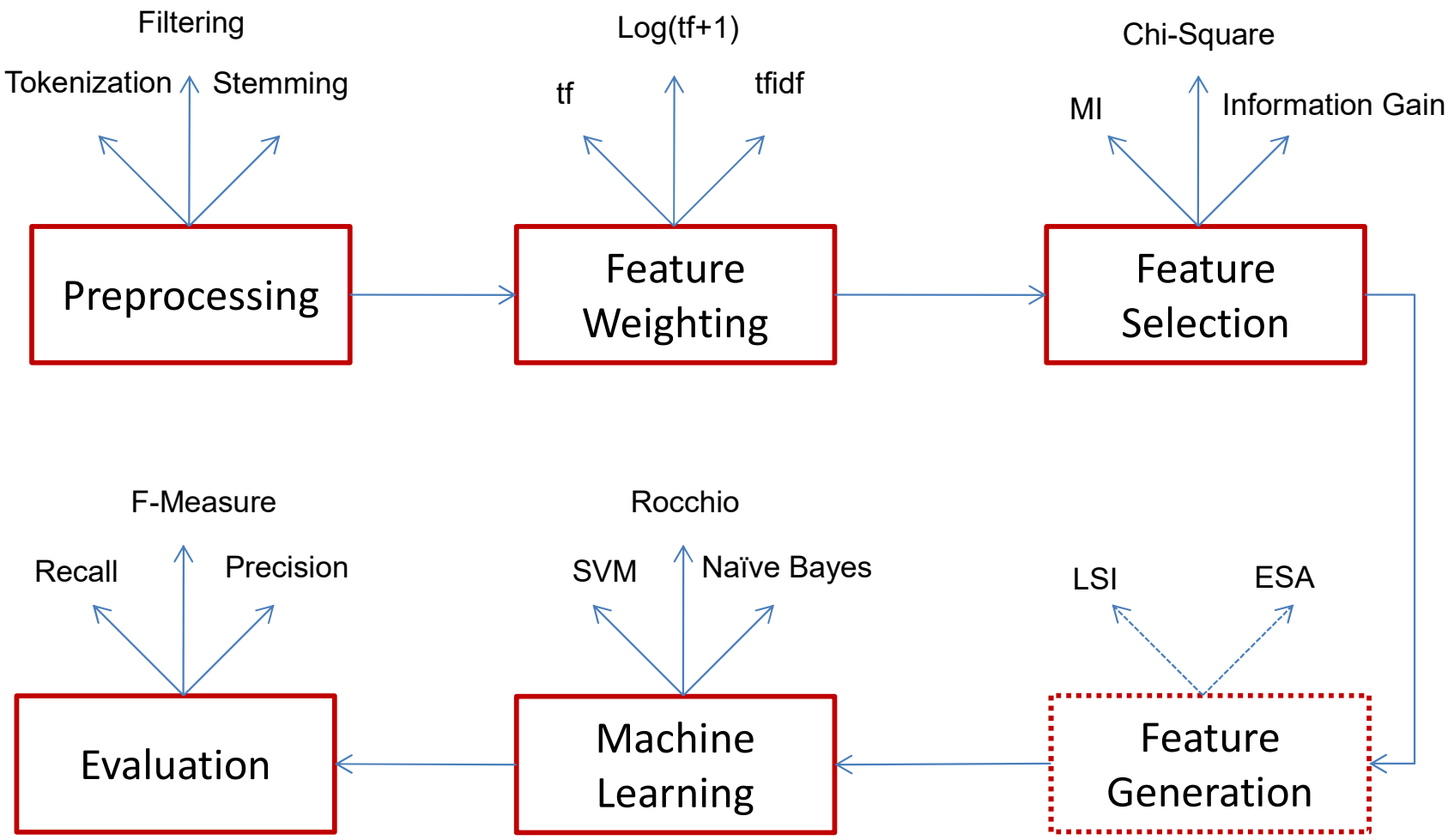
- a learned classifier $\gamma:d \rightarrow c$

Unsupervised Learning

- No labeled examples
- The system tries to cluster documents based on some heuristics & distance measures

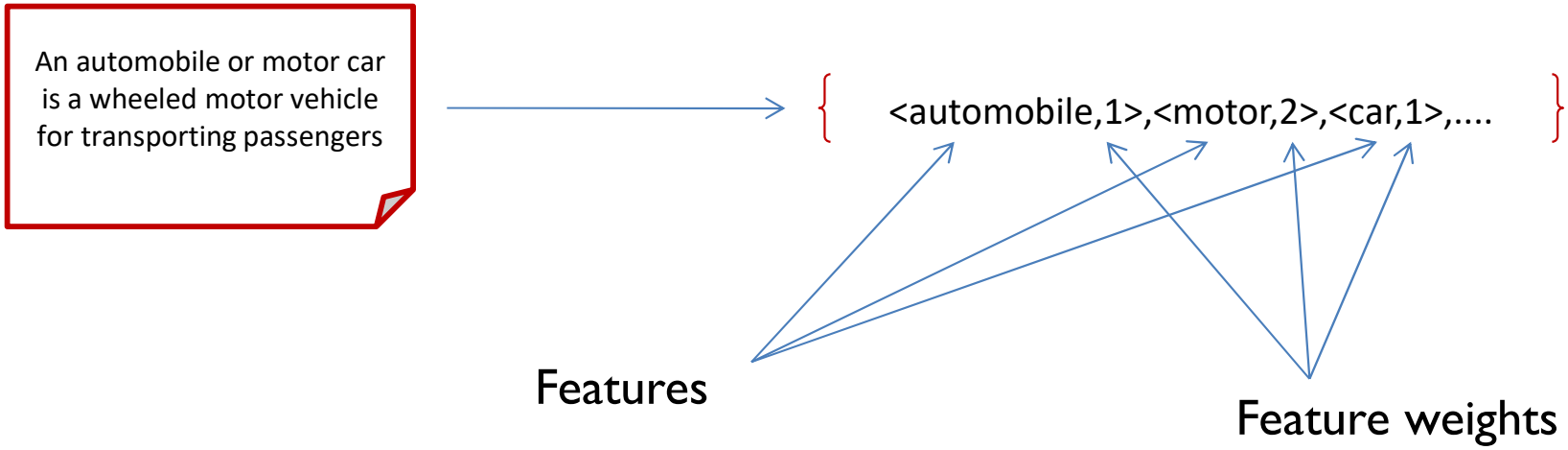


Framework of a Classical Text Categorizer



Document Representation

- The idea is to process the natural language text in a document and transform into a vector



- Document Representation is a vector of term weights
- Each term represents specific information about the original document. Terms are sometimes referred to as features
- Each term usually has an associated weight which represents its contribution to the document

Terms

- Simplest approach: a term is a word
  - Bag of Words
- Preprocessing
  - Stop word removal (“a”,”the”,”of”,”and”)
  - Stemming (“stemming”,”stemmed”,”stemmer”)
- Ignore word order
- Sophisticated Approaches
- Higher Order statistics
- Phrases (how to define?)
  - Syntactically – according to grammar (Noun phrases)
  - Statistically – strongly occurring patterns of words

Weights

- Term frequency $$tf(t_k) = \sum_i \frac{n_k}{n_i}$$
- TF-IDF
  - The more often the term appears in a document, the more the representative is it of the document.
  - The more documents the term appears in the less discriminating it is.

$$tf.idf(t_k) = tf(t_k).idf(t_k) \quad idf(t_k) = \log(\frac{N_i}{N_k})$$

- Normalized TF-IDF
  - Normalize the tf.idf values to the range[0,1]

$$w(t_k) = \frac{tf.idf(t_k)}{\sqrt{\sum_{s=1} (tf.idf(t_s))^2}}$$

Dimensionality Reduction

- There are many terms
  - Many learning algorithms don't deal with extremely high dimensions
  - Over fitting problem
  - Not all terms are equally effective
- Dimensionality Reduction - Feature Selection
  - Idea: find a more efficient document representation, with much fewer dimensions, with a minimal loss of effectiveness (accuracy).
- Local vs. Global Policies
  - Local Policy: For each category, find the best terms.
  - Global Policy: Given all the categories find the best terms.

Term Selection

- Out of original set of terms, *t*, find a much smaller subset, *t'*, that yields high-test effectiveness (accuracy).

- Examples
  - Chi Square
  - Mutual Information
  - Information Gain
  - Information Ratio
  - Odd Ratio

Feature Generation

- Term Clustering
  - Unsupervised
  - Supervised
  - Distributional clustering

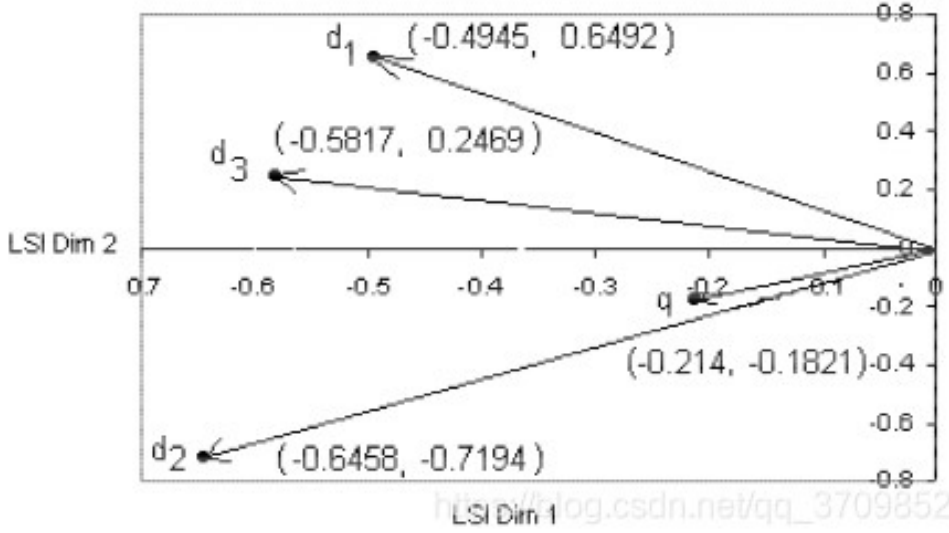
- Latent Semantic Indexing
- Explicit Semantic Indexing

Latent Semantic Indexing (LSI)

- Words by themselves are not a good measure.
  - Synonyms (car, automobile)
  - Polysemous (Apple, Jaguar)
- LSI: a method for inferring the contextual similarity of terms
  - Finds the best *m* uncorrelated terms that best describe the original *n* terms.
  - Uncover latent information (synonyms)

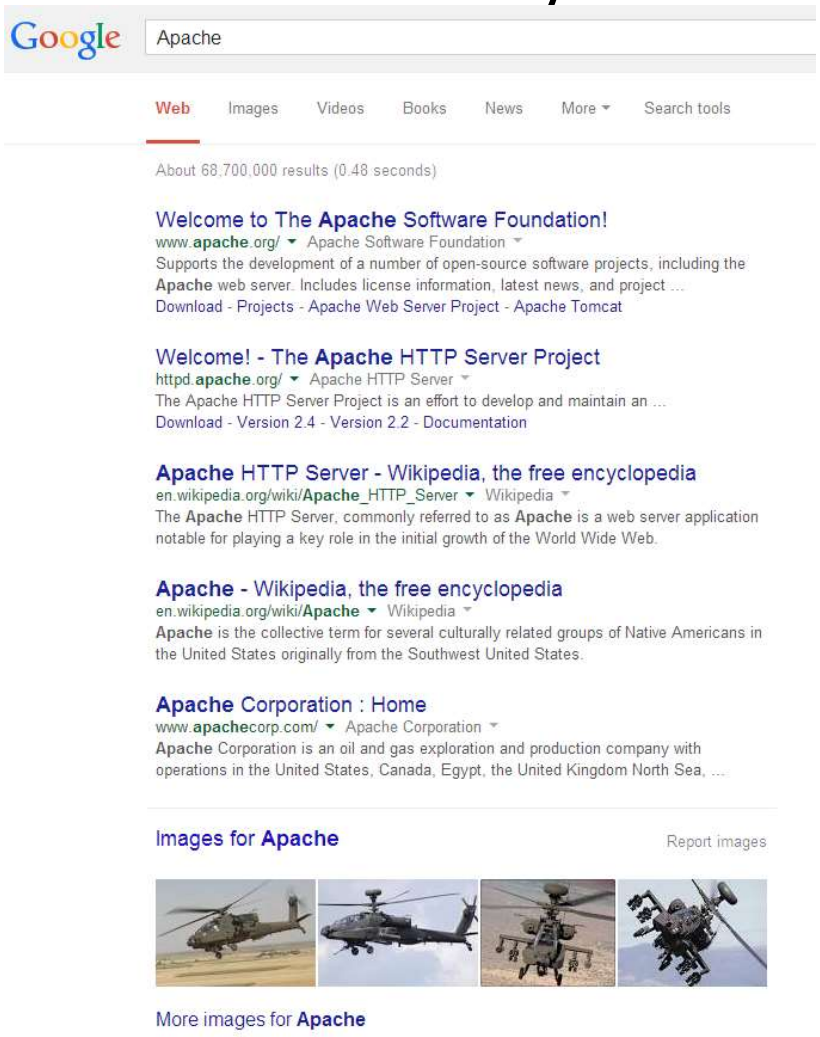
Latent Semantic Indexing (LSI)

- Algorithm
  - Calculate term weights and construct word-document matrix A and query matrix
  - Decompose matrix A and find U, S, and $A = USV^T$
  - Reduce rank U, S , V
  - Find the new document vector and the new query vector
  - Sort documents in descending order of query-document cosine similarity



Latent Semantic Indexing (LSI)

- can use character strings to determine the semantics of text – what that the text actually means



Cons: Latent Semantic Indexing (LSI)

- leverages word co-occurrence information from a large unlabeled corpus of text;
- does not use any explicit human-organized knowledge;
- identifies a number of most prominent dimensions in the data, which are assumed to correspond to “latent concepts”

Explicit Semantic Analysis (ESA)

- Get a basic concepts through wiki: C_1, C_2,…,C_n.

- the text *t* of any length is expressed as the weight vector w_1, w_2,..., w_n with the same length as the above vector, respectively, indicating the degree of correlation between t and the concept Ck.

Explicit Semantic Analysis (ESA)

- Bag of Words

{ American politics } → Democrats, Republicans, abortion, taxes, homosexuality, guns, etc

- Explicit Semantic Analysis

{ Car } → Wikipedia:Car, Wikipedia:Automobile, Wikipedia:BMW, Wikipedia:Railway, etc

More Discussions on Feature Selection and Generation

- Finding appropriate feature set for specific categorization task
  - Granularity
    - Chinese character / Chinese Word
    - Punctuation
    - Pattern
  - Specific task
    - "The *Dream* of the Red Chamber" Cao Xueqin or Gao Er
      - Use stopwords
    - Sentiment Analysis
      - Sentiment oriented words
      - Sentiment words

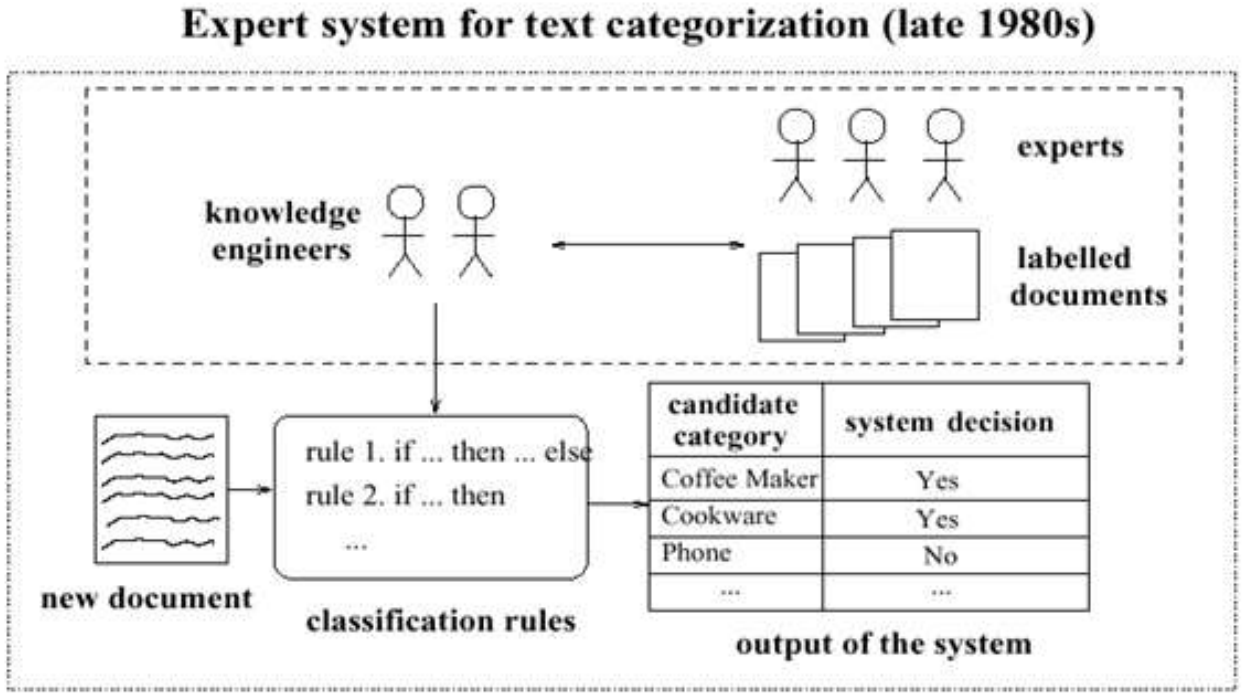
Rule-based Approach to Text Categorization

- Text in a Web Page

  "Saeco revolutionized *espresso* brewing a decade ago by introducing Saeco SuperAutomatic *machines*, which go from bean to *coffee* at the touch of a button. The all-new Saeco Vienna Super-Automatic home coffee and *cappucino machine* combines top quality with low price!"

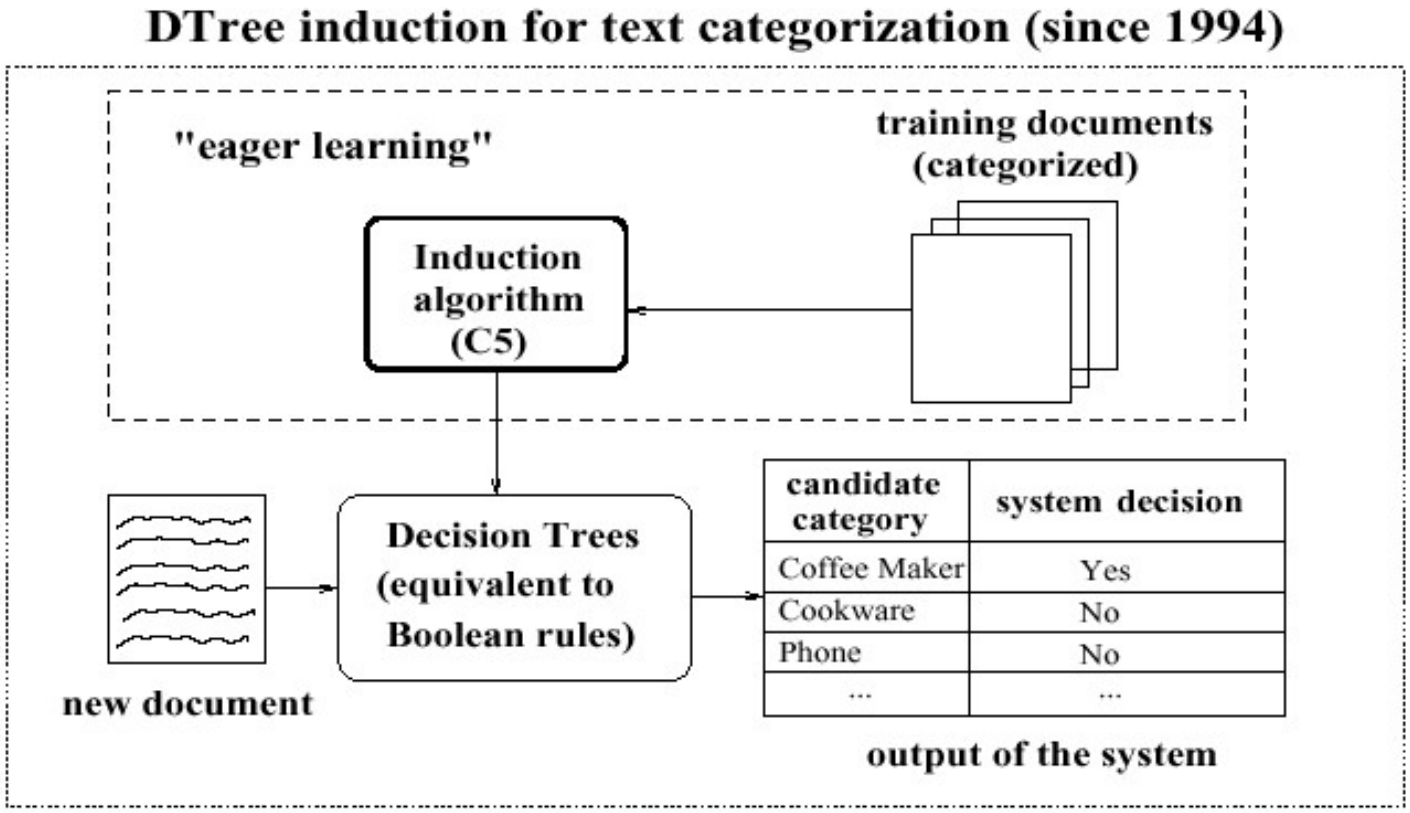
- Rules
  - Rule 1.
    (*espresso* **or** *coffee* **or** *cappucino*) **and** *machine** → *Coffee Maker*
  - Rule 2.
    *automat** **and** *answering* **and** *machine** → *Phone*
  - Rule ...

Expert System for Text Categorization (late 1980s)



- Experience has shown
  - too time consuming
  - too difficult
  - inconsistency issues (as the rule set gets large)

Replace Knowledge Engineering with a Statistical Learner



Knowledge Engineering vs. Statistical Learning

- For US Census Bureau Decennial Census 1990
  - 232 industry categories and 504 occupation categories
  - $15 million if fully done by hand
- Define classification rules manually:
  - Expert System AIOCS
  - Development time: 192 person-months (2 people, 8 years)
  - Accuracy = 47%
- Learn classification function
  - Nearest Neighbor classification (Creecy ’92: 1-NN)
  - Development time: 4 person-months (Thinking Machine)
  - Accuracy = 60%

Machine Learning-based Methods

- Rocchio
- Naive Bayesian
- K-Nearest Neighbour
- SVM
- Decision Tree

Using Relevance Feedback (Rocchio)

- Relevance feedback methods can be adapted for text categorization.
- Use standard TF/IDF weighted vectors to represent text documents (normalized by maximum term frequency).
- For each category, compute a *prototype* vector by summing the vectors of the training documents in the category.
- Assign test documents to the category with the closest prototype vector based on cosine similarity

Rocchio Text Categorization Algorithm (Training)

Assume the set of categories is $\{c_1, c_2, \ldots c_n\}$

For i from 1 to n let $\mathbf{p}_i = <0, 0, \ldots, 0>$ (*init. prototype vectors*)

For each training example $<x, c(x)> \in D$

a. Let **d** be the frequency normalized TF/IDF term vector for doc x
b. Let $i = j: (c_j = c(x))$ (*sum all the document vectors in c_i to get $\mathbf{p}_i$*)
c. Let $\mathbf{p}_i = \mathbf{p}_i + \mathbf{d}$

Rocchio Text Categorization Algorithm (Test)

Given test document *x*
Let **d** be the TF/IDF weighted term vector for *x*
Let $m = -2$ (*init. maximum cosSim*)
For *i* from 1 to *n*:(*compute similarity to prototype vector*)

a. Let $s = \text{cosSim}(\mathbf{d}, \mathbf{p}_i)$
b. if $s > m$
    I. let $m = s$
    II. let $r = c_i$ (*update most similar class prototype*)

Return class *r*

Rocchio Properties

- Does not guarantee a consistent hypothesis.
- Forms a simple generalization of the examples in each class (a *prototype*).
- Prototype vector does not need to be averaged or otherwise normalized for length since cosine similarity is insensitive to vector length.
- Classification is based on similarity to class prototypes.

Bayesian for Text Classification

- Maximum a posteriori classification
  - predict the class c that has the highest probability given the document D $c = \arg\max_c p(c|\mathbf{d})$
- Problem:
  - we have not seen the document often enough to directly estimate p(c|d)
- Bayes Theorem: $p(c|\mathbf{d}) \cdot p(\mathbf{d}) = p(\mathbf{d}|c) \cdot p(c)$
- p(d) is only for normalization: $p(\mathbf{d}) = \sum_c p(\mathbf{d}|c)\, p(c)$
- Bayes Classifier:

$$c = \arg\max_c \; p(\mathbf{d}|c)\, p(c)$$

If all prior probabilites p(c) are identical → maximum likelihood prediction

Bayesian for Text Classification

- a document is a sequence of n terms: $p(\mathbf{d}|c)=p(t_1,t_2,....t_n|c)$

$$p(\mathbf{d}|c)=\prod_{i=1}^{|\mathbf{d}|} p(t_i|c)$$

- Apply Independence Assumption:
  - $p(t_i|c)$ is the probability with which the word occurs in documents of class c

- Naïve Bayes Classifier:

$$c=\arg\max_c \prod_{i=1}^{|\mathbf{d}|} p(t_i|c)\,p(c)$$

Bayesian for Text Classification

- Estimating Probabilities
  - estimate probabilities from the frequencies in the training set: $p(t_i=w|c)=\frac{n_{w,c}}{\sum_{w\in W} n_{w,c}}$
  - word w occurs n(d,w) times in document d: $n_{w,c}=\sum_{\mathbf{d}\in c} n(\mathbf{d},w)$
- Problem:
  - test documents may contain new words
  - those will be have estimated probabilities 0
  - assigned probability 0 for all classes
- Smoothing of probabilities:
  - basic idea: assume a prior distribution on word probabilities:

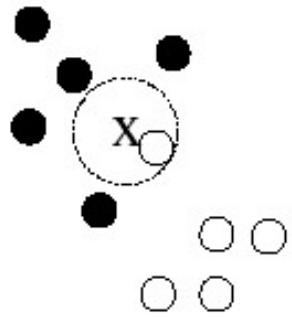
$$p(t_i=w|c)=\frac{n_{w,c}+1}{\sum_{w\in W}(n_{w,c}+1)}=\frac{n_{w,c}+1}{\sum_{w\in W} n_{w,c}+|W|}$$

Nearest-Neighbor Learning Algorithm

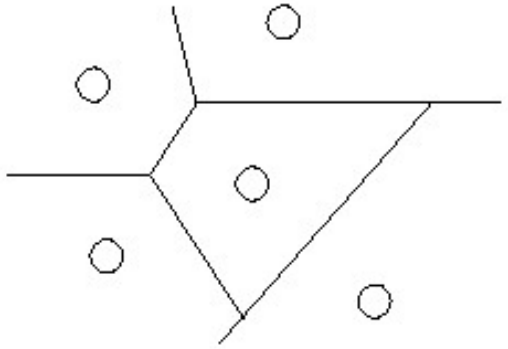
- Learning is just storing the representations of the training examples in *D*.
- Initially by Fix and Hodges (1951)
- Theoretical error bound analysis by Duda & Hart (1957)
- Testing instance *x*:
  - Compute similarity between *x* and all examples in *D*.
  - Assign *x* the category of the most similar example in *D*.
- Does not explicitly compute a generalization or category prototypes.
- Also called:
  - Case-based Memory-based Lazy learning

1-Nearest Neighbor

1-NN: assign "x" (new point) to the class of it nearest neighbor

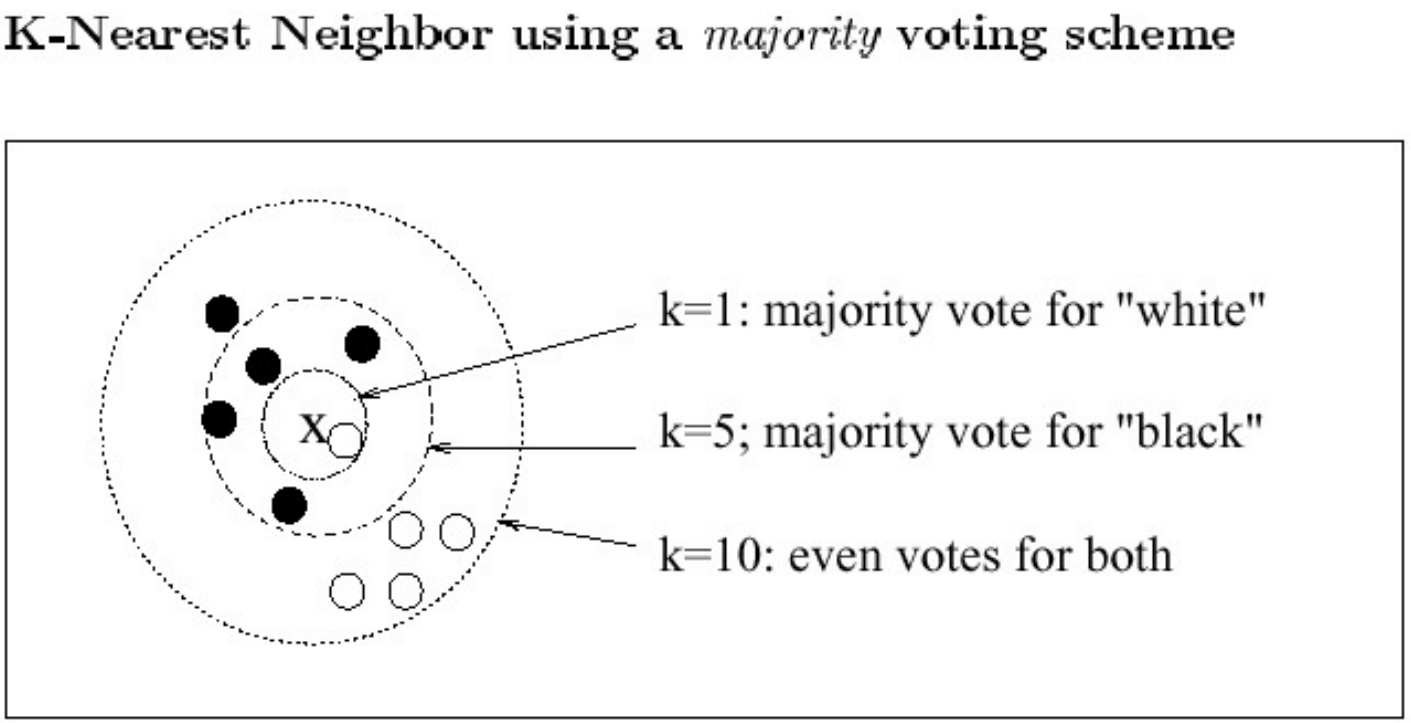


assign "x" to "white"



decision surface divided by points ("Voronoi diagram")

K-Nearest Neighbor using a *majority* voting scheme



Category Scoring for Weighted-Sum

- The score for a category is the sum of the similarity scores between the point to be classified and all of its k-neighbors that belong to the given category.

- To restate: $$score(c \mid x) = \sum_{d \in kNN\ of\ x} sim(x,d)\, I(d,c)$$

  where *x* is the new point; *c* is a class (*e.g. black* or *white);*
  d is a classified point among the k-nearest neighbors of *x*;
  sim(x,d) is the similarity between *x* and *d;*
  I(d,c) = 1 iff point *d* belongs to class *c*;
  I(d,c) = 0 otherwise.

K Nearest Neighbor for Text Categorization (Yang, SIGIR-1994)

- Represent documents as points (vectors).
- Define a similarity measure for pairwise documents.
- Tune parameter *k* for optimizing classification effectiveness.
- Choose a voting scheme (e.g., weighted sum) for scoring categories
- Threshold on the scores for classification decisions.

K Nearest Neighbor for Text Categorization

Training:

For each training example $\langle x, c(x) \rangle \in D$

Compute the corresponding TF-IDF vector, $\mathbf{d}_x$, for document x

Test instance y:

Compute TF-IDF vector $\mathbf{d}$ for document y

For each $\langle x, c(x) \rangle \in D$

Let $s_x = \text{cosSim}(\mathbf{d}, \mathbf{d}_x)$

Sort examples, x, in D by decreasing value of s_x

Let N be the first k examples in D. (*get most similar neighbors*)

Return the majority class of examples in N

Similarity Measures

- Cosine similarity

$$\cos(\vec{x}, \vec{y}) = \frac{\sum_i x_i y_i}{\sqrt{\sum_i x_i^2} \times \sqrt{\sum_i y_i^2}}$$

- Simplest for continuous *m*-dimensional instance space is *Euclidian distance*
- Simplest for *m*-dimensional binary instance space is *Hamming distance* (number of feature values that differ)
- Kullback-Leibler distance (distance between two probability distributions)
- For text, cosine similarity of TF-IDF weighted vectors is typically most effective

Key Components in kNN

- Functional definition of “similarity”
  - e.g. cos, Euclidean, kernel functions, …
- How many neighbors do we consider?
  - Value of *k* determined *empirically* (see methodology section)
- Does each neighbor get the same weight?
  - Weighted-sum or not
- All categories in neighborhood? Most frequent only? How do we make the final decision?
  - Rcut, Pcut, or Scut

Pros of kNN

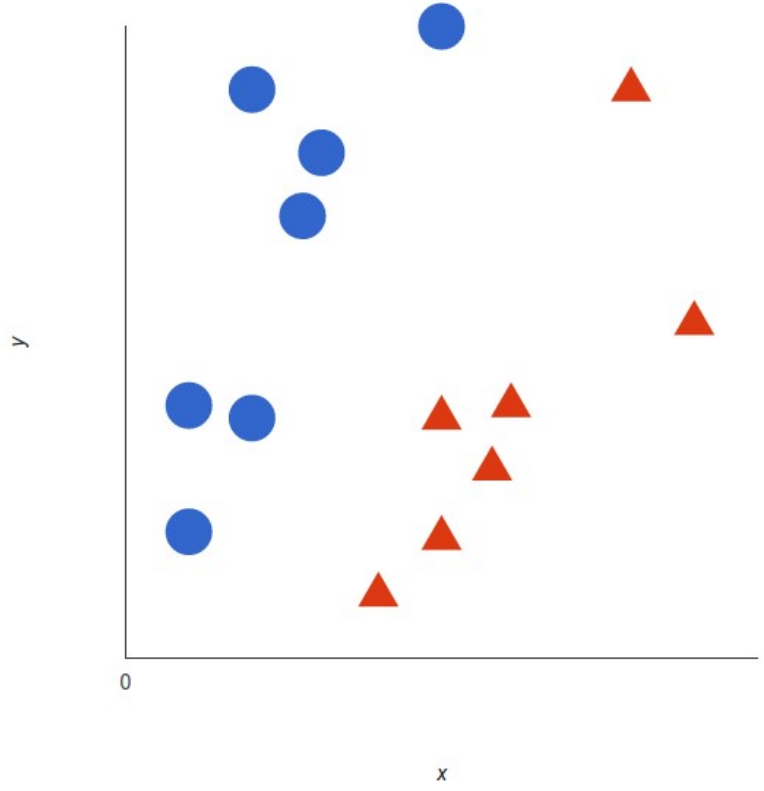
- Simple and effective (among top-5 in benchmark evaluations)
  - Non-linear classifier (vs linear)
  - Local estimation (vs global)
  - Non-parametric (very few assumptions about data)
  - Reasonable similarity measures (borrowed from IR)
- Computation (time & space) linear to the size of training data
- Low cost for frequent re-training, i.e., when categories and training documents need to be updated (common in Web environment and e-commerce applications)

Cons of kNN

- Online response is typically slower than *eager learning* algorithms
  - Trade-off between off-line training cost and online search cost
- Scores are not normalized (probabilities)
  - Comparing directly to and combining with scores of other classifiers is an open problem
- Output not good in explaining why a category is relevant
  - Compared to DTree, for example (take this with a grain of salt).

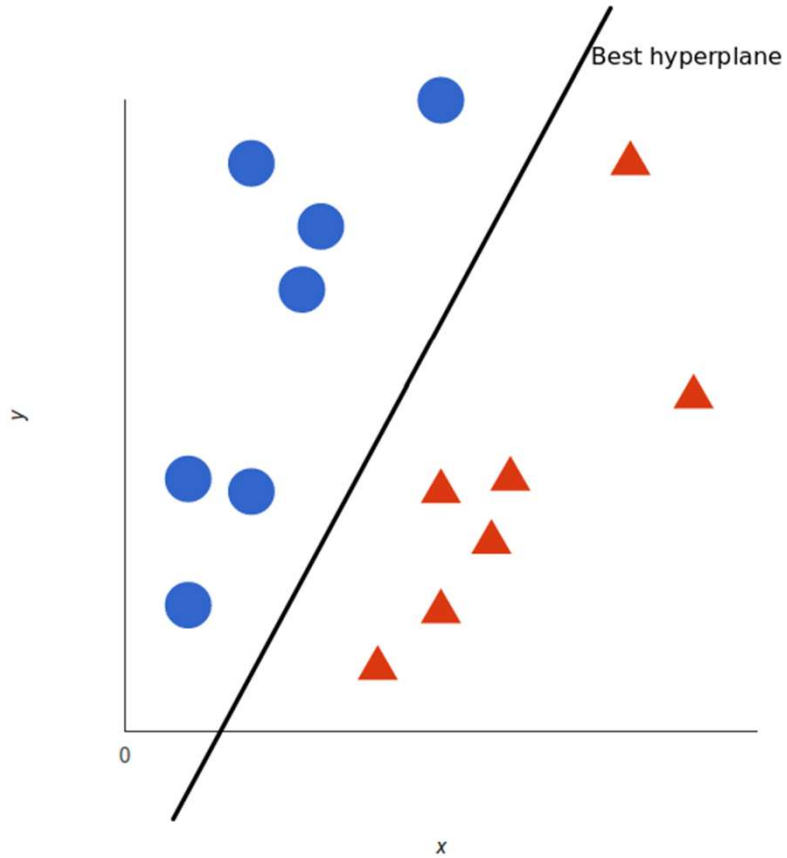
SVM for Text Categorization

- two categories: red and blue.
- data has two characteristics: x and y.
- a classifier, given a pair of (x, y) coordinates, the output is limited to red or blue.



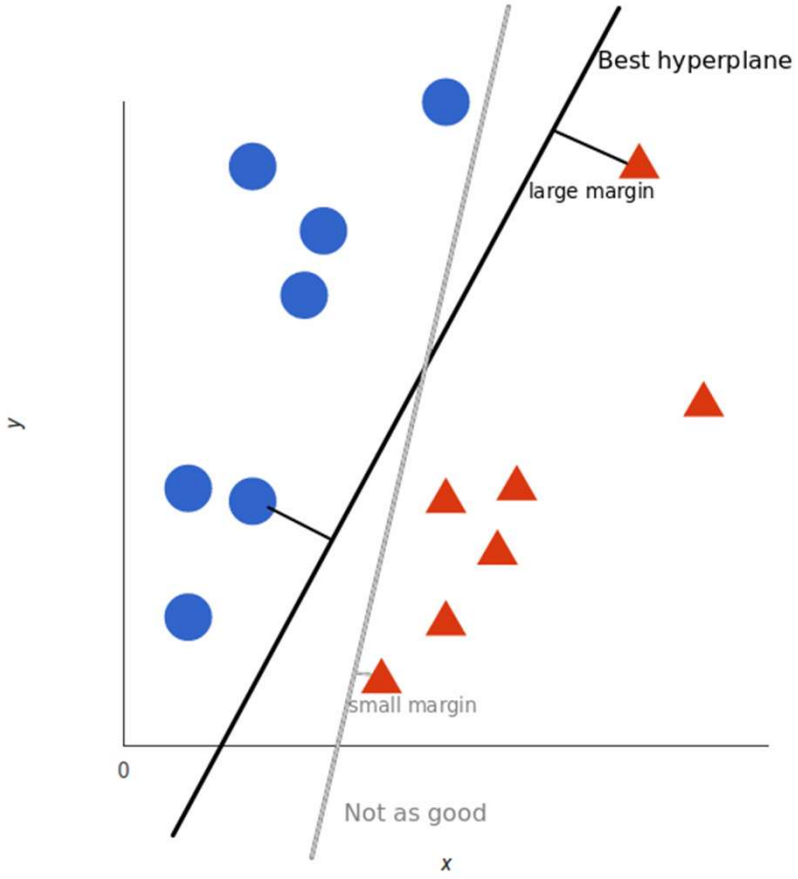
SVM for Text Categorization

- SVM accepts these data points and outputs a hyperplane to separate the two categories.
- This line is the decision boundary: separate red and blue.



SVM for Text Categorization

- The best hyperplane (in this case, a line) is the furthest away from the closest element in each category.



SVM for Text Categorization

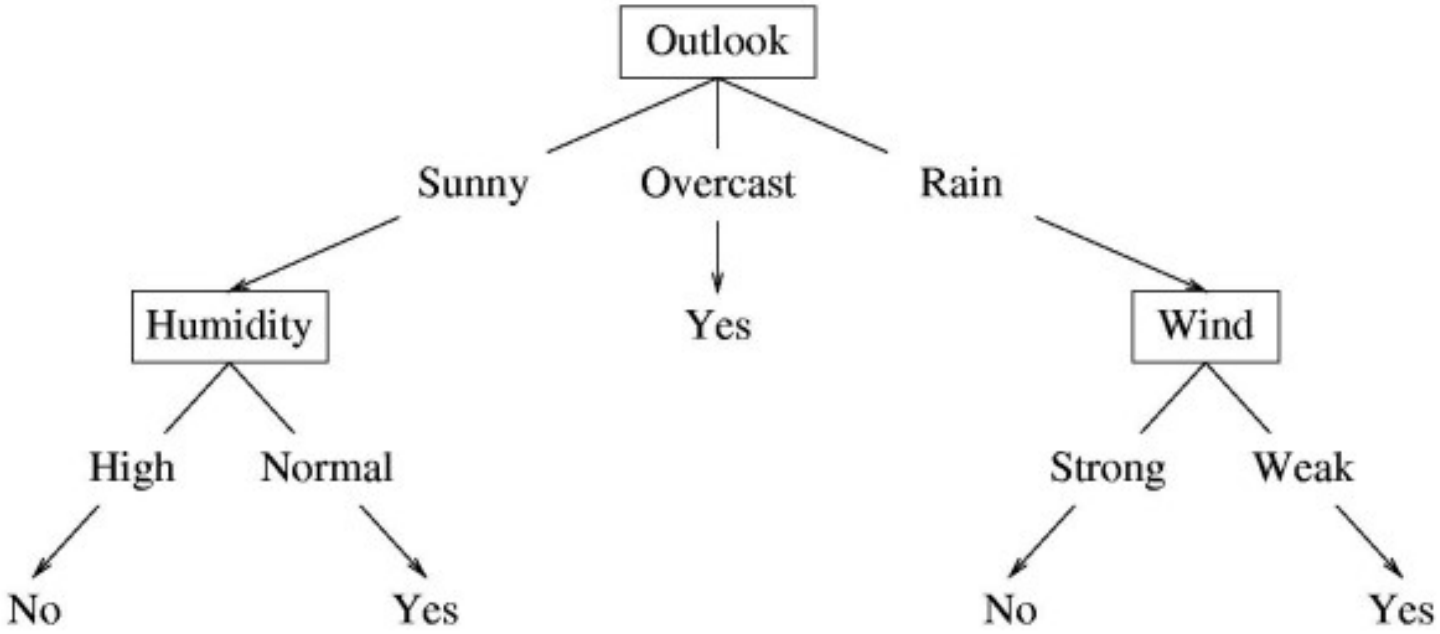
- integrate the text into a vector.
  - word frequency. This means that the text is regarded as a bag of words.
  - TF-IDF. For problems with higher computational requirements, we can also use TF-IDF.
- each sample is represented by a vector of several thousand.
- preprocessing techniques to further improve its effects, such as stop word removal, and n-grams.

SVM for Text Categorization

- Pros:
  - a convex optimization problem.
  - data with high-dimensional sample space.
  - the theoretical foundation is relatively complete

- Cons:
  - involve the calculation of the m-order matrix, so SVM is not suitable for very large data sets.
  - only applicable to two classification problems.

Decision Trees for Text Categorization



- Each internal node: test one attribute X_i
- Each branch from a node: selects one value for X_i
- Each leaf node: predict Y

Decision Trees Algorithm

- node = root of decision tree
  - Main loop:
    - A ←the “best” decision attribute for the next node.
    - Assign A as decision attribute for node.
    - For each value of A, create a new descendant of node.
    - Sort training examples to leaf nodes.
    - If training examples are perfectly classified, stop. Else, recurse over new leaf nodes.

Decision Trees Algorithm

- **Key problem:** choosing which attribute to split a given set of examples
  - Some possibilities are:
    - Random: Select any attribute at random
    - Least-Values: Choose the attribute with the smallest number of possible values
    - Most-Values: Choose the attribute with the largest number of possible values
    - Max-Gain: Choose the attribute that has the largest expected information gain
- The ID3 algorithm uses the Max-Gain method of selecting the best attribute

Decision Trees for Text Categorization

- First vectorizing the corpus stored in the “text” column;
- Then apply TF-IDF to get features;
- and finally build the decision trees;

Decision Trees for Text Categorization

- Pros:
  - interpret and visualize
  - fewer data preprocessing from the user
  - feature engineering such as predicting missing values

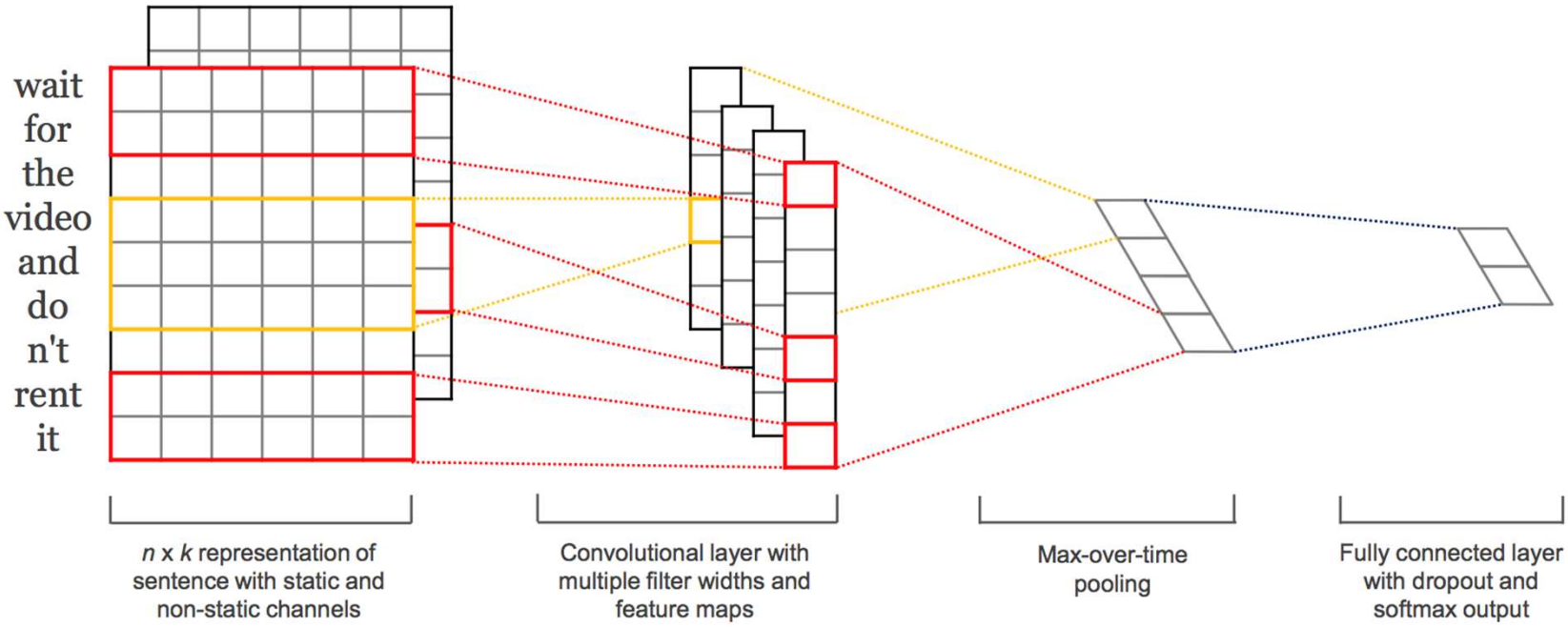
- Cons:
  - sensitive to noisy data
  - the small variation(or variance) in data can result in the different decision tree

Deep Learning-based Methods

- CNN
- Char-CNN
- Deep Pyramid CNN
- RCNN

CNN for Text Categorization (Kim EMNLP-2014)

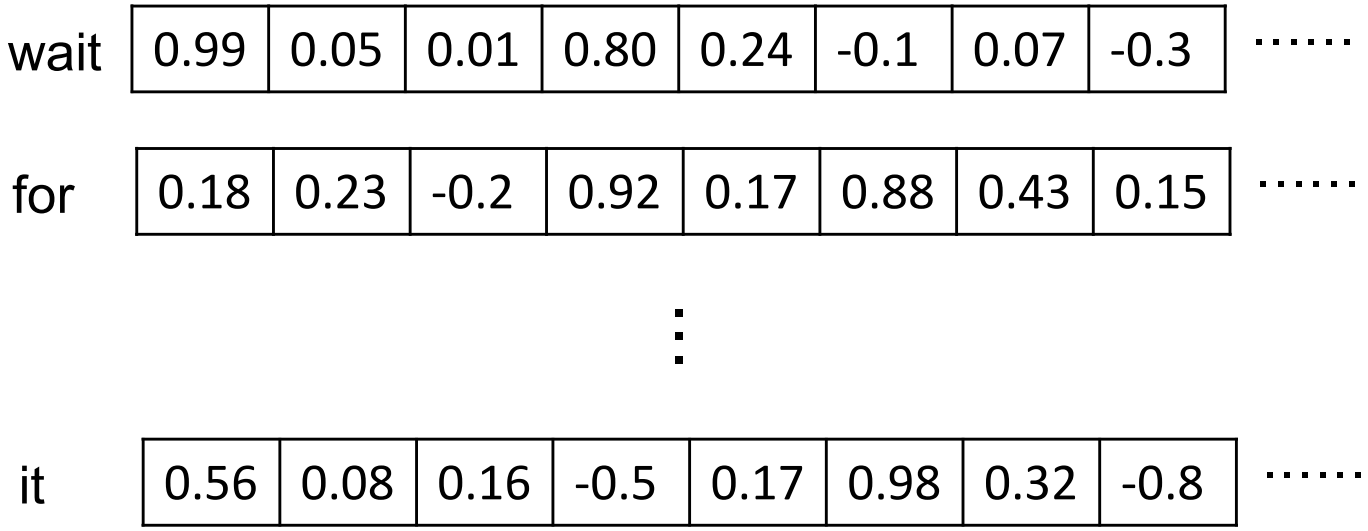
The model architecture for text categorization is shown below:



Embedding layer

Let $x_i \in R^k$ be the k-dimensional word vector corresponding to the i-th word in the sentence. A sentence of length n (padded where necessary) is represented as:

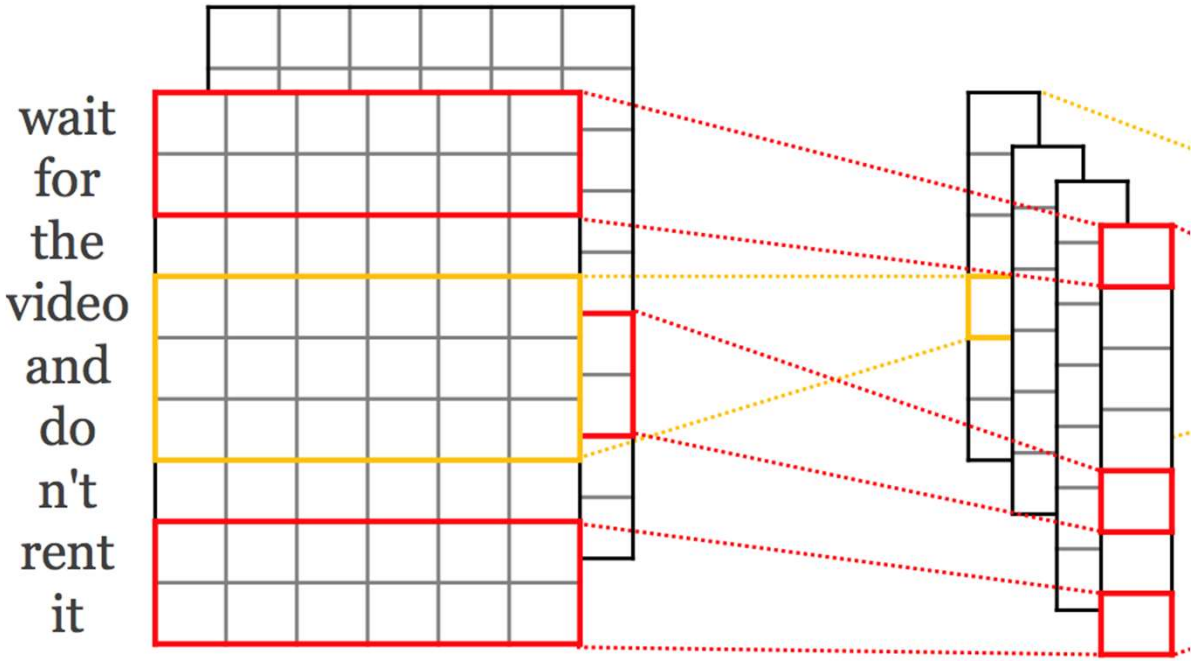
$$x_{1:n} = x_1 \oplus x_2 \oplus \cdots \oplus x_n$$



Convolutional layer

A convolution operation involves a filter $w \in R^{hk}$, which is applied to a window of h words to produce a new feature. For example, a feature c_i is generated from a window of words $x_{i:i+h-1}$ by：

$$c_i = f(w \cdot x_{i:i+h-1} + b)$$



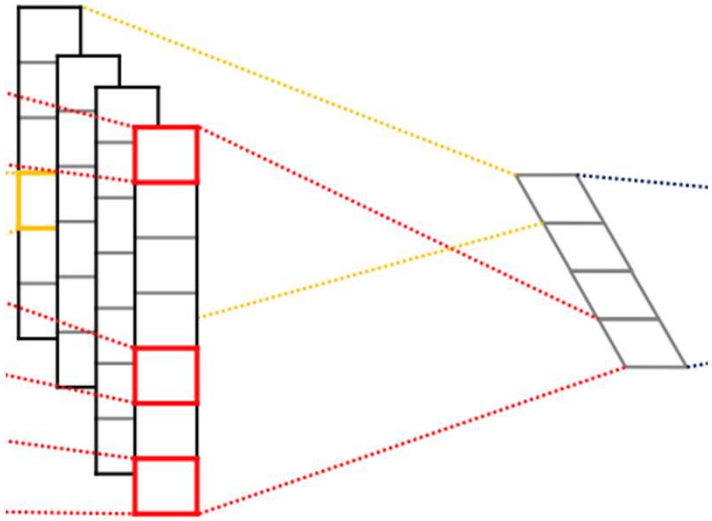
Pooling layer

This filter is applied to each possible window of words in the sentence $\{x_{1:h}, x_{2:h+1}, \ldots, x_{n-h+1:n}\}$ to produce a feature map：

$$c = [c_1, c_2, \cdots, c_{n-h+1}]$$

Then apply a max-overtime pooling operation over the feature map and take the maximum value：

$$\hat{c} = \max\{c\}$$



Regularization

Given the penultimate layer $z = [\hat{c}_1, \ldots, \hat{c}_m]$, for output unit *y* in forward propagation, dropout uses:

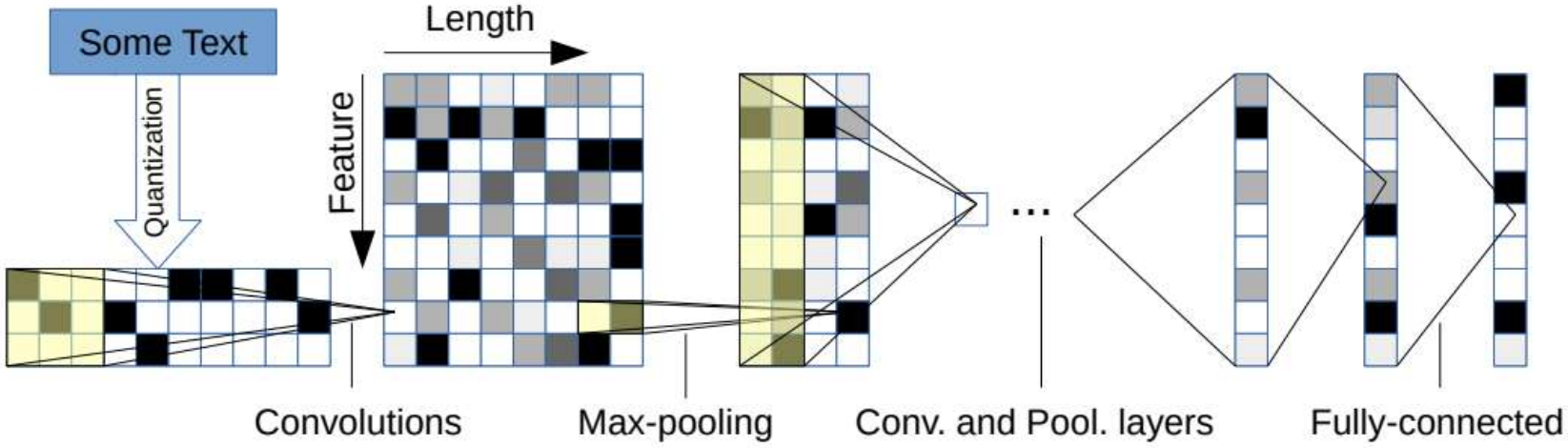
$$y = w \cdot (z \circ r) + b$$

At test time, the learned weight vectors are scaled by p such that $\hat{w} = pw$, and $\hat{w}$ is used (without dropout) to score unseen sentences.

Additionally constrain l_2-norms of the weight vectors by rescaling w to have $||w||_2 = s$ whenever $||w||_2 > s$ after a gradient descent step.

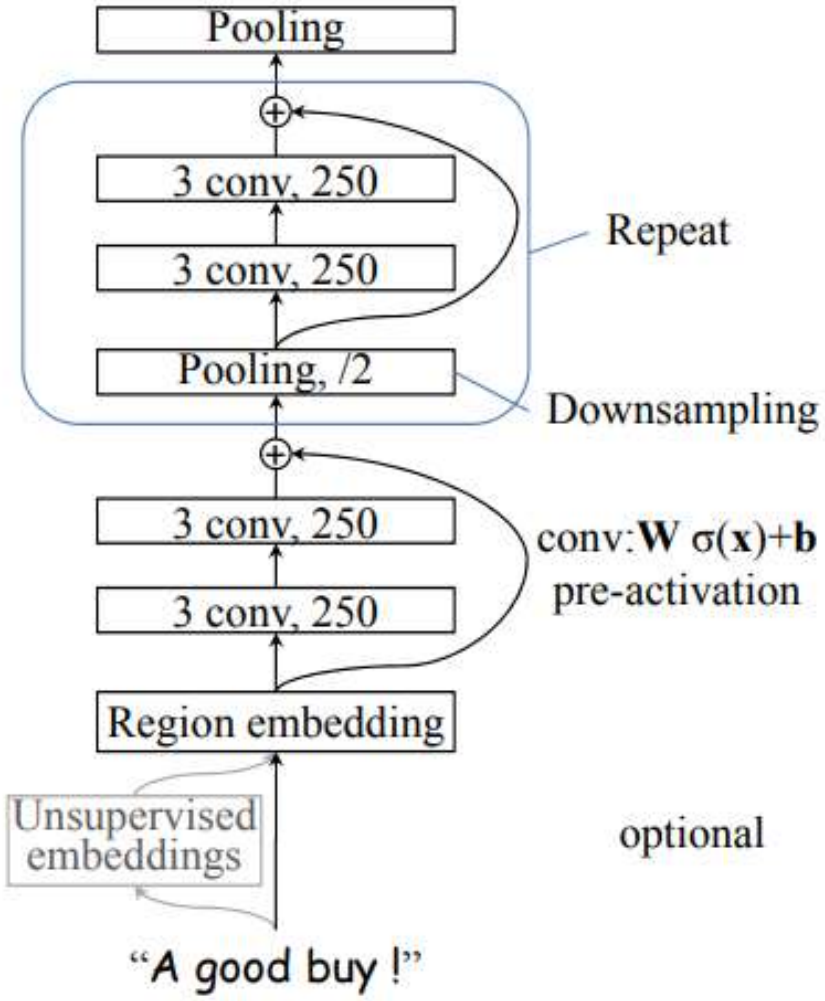
Character-level Convolutional Networks for Text Classification (*Zhang et al., NIPS 2015*)

Character-level ConvNet is an effective method



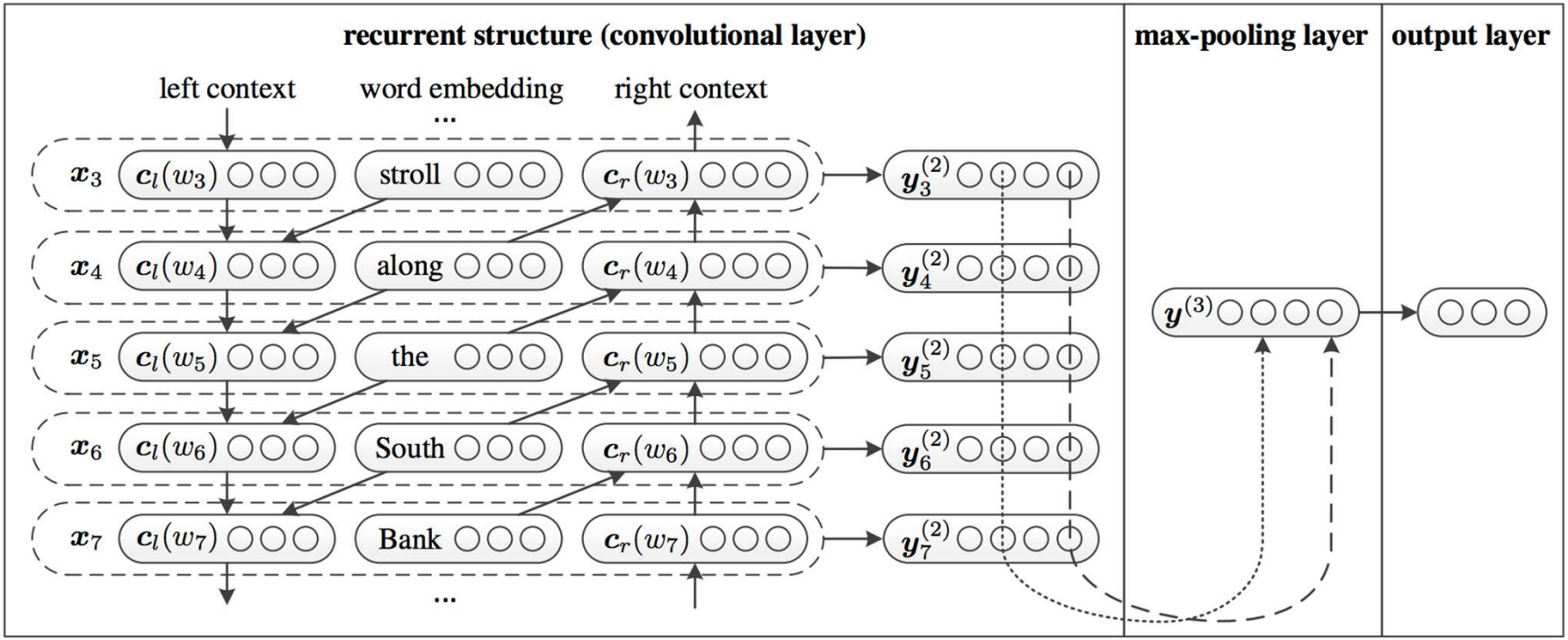
Deep Pyramid Convolutional Neural Networks for Text Categorization (*Johnson et al.,ACL 2017*)

- Region embedding + shortcut connection



Recurrent Convolutional Neural Networks for Text Classification (*Lai et al., AAAI 2015*)

- BiLSTM + Max Pooling



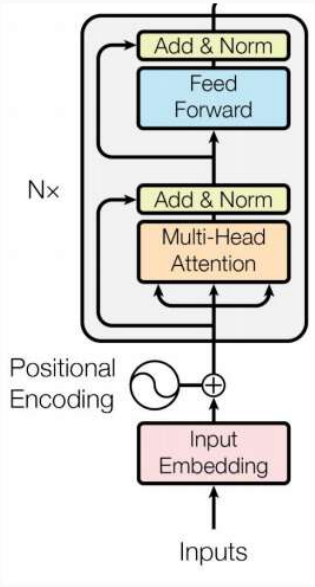
Pretraining Model based Methods

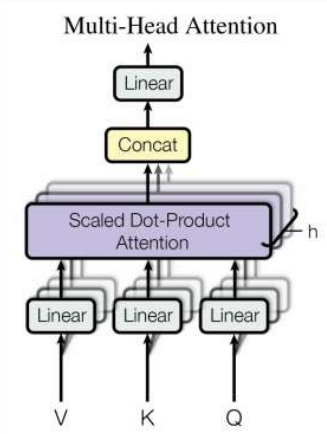
- BERT
- BERT for text classification
- RoBERTa
- T5

Bert: Pre-training of deep bidirectional transformers for language understanding (*Devlin et al., HLT-NAACL 2018*)

Transformer

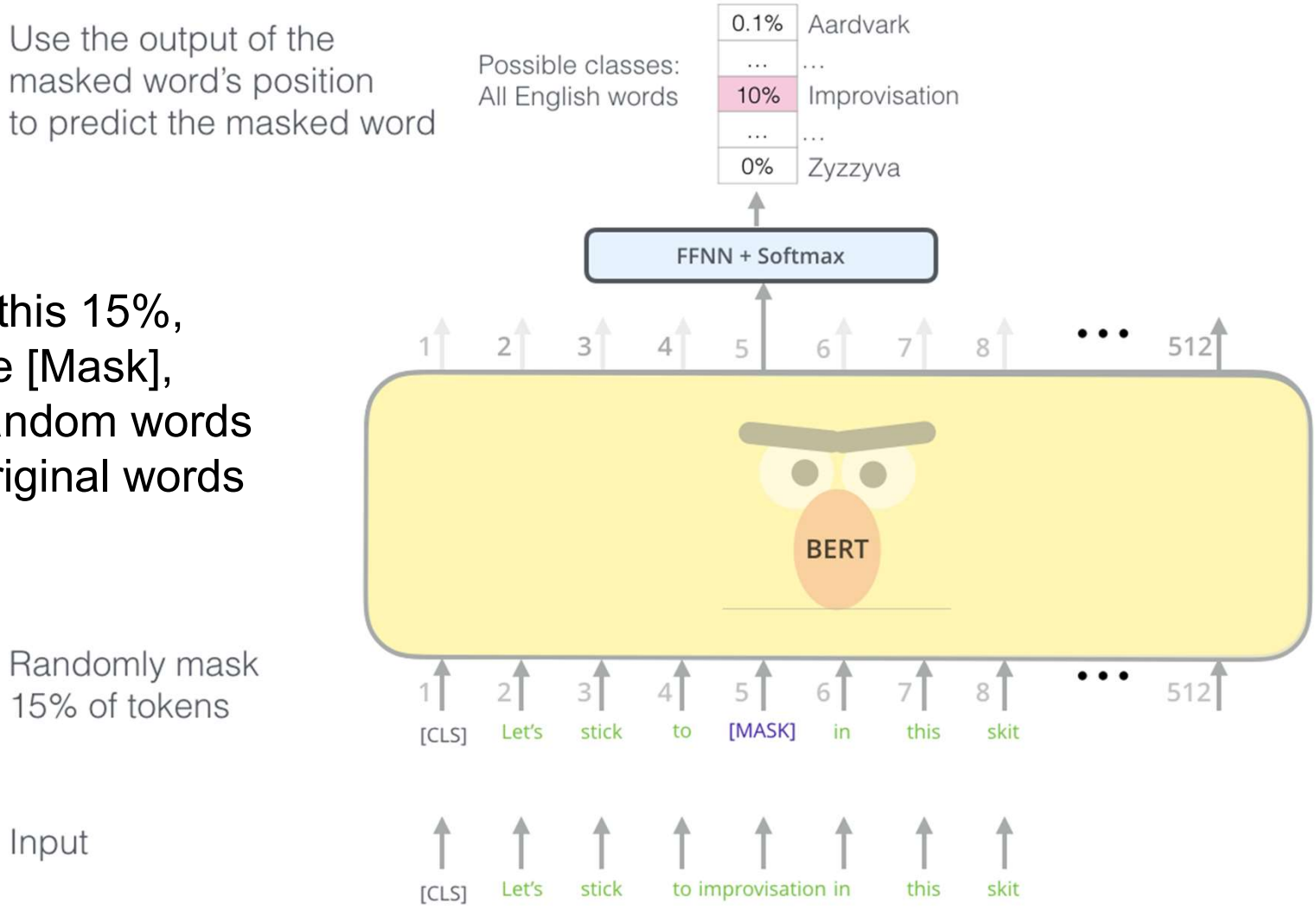
- Multi-headed self attention
  - Models context
- Feed-forward layers
  - Computes non-linear hierarchical features
- Layer norm and residuals
  - Makes training deep networks healthy
- Positional embeddings
  - Allows model to learn relative positioning



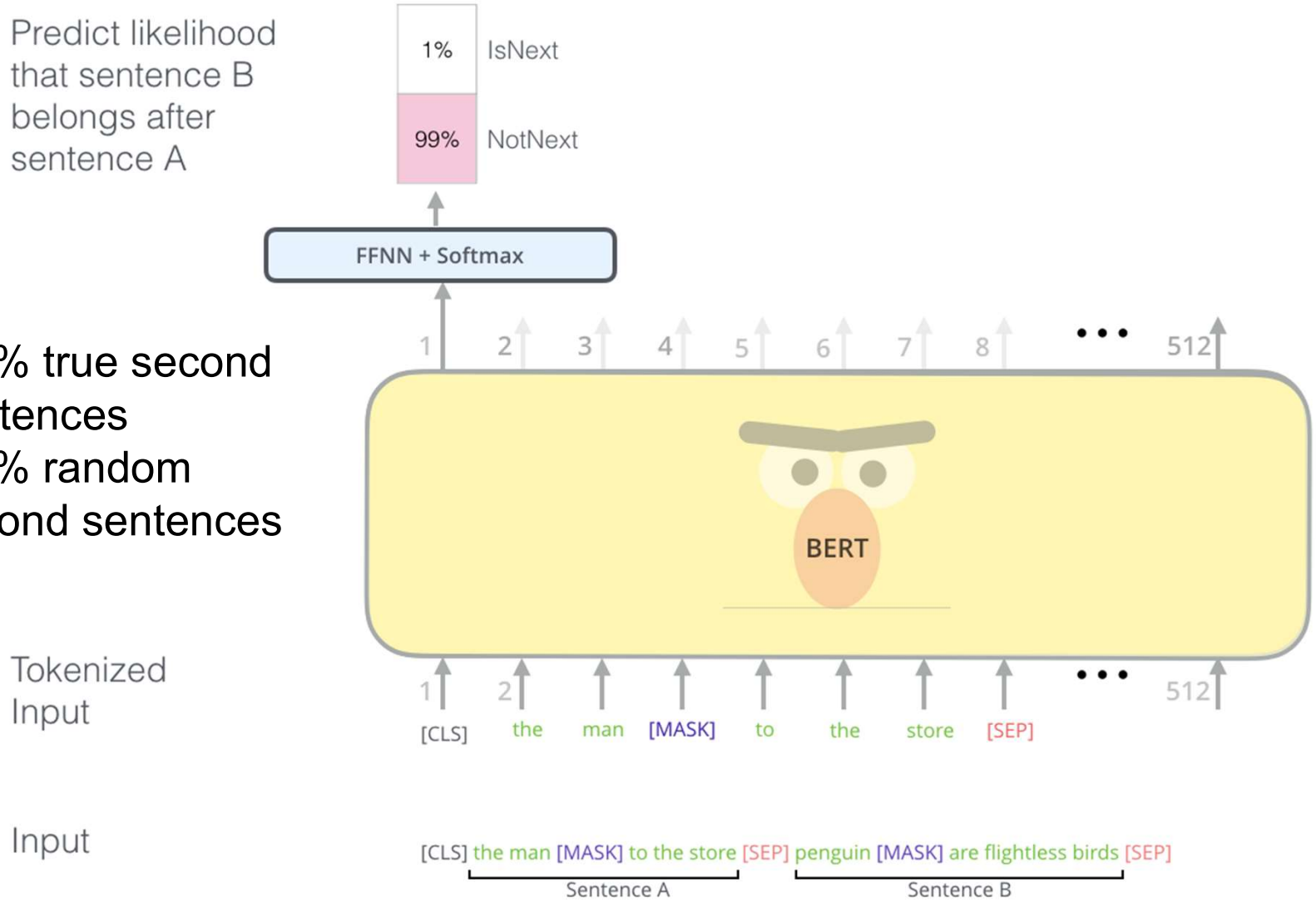


Bert: Pre-training of deep bidirectional transformers for language understanding (*Devlin et al., HLT-NAACL 2018*)

-Out of this 15%,
80% are [Mask],
-10% random words
-10% original words



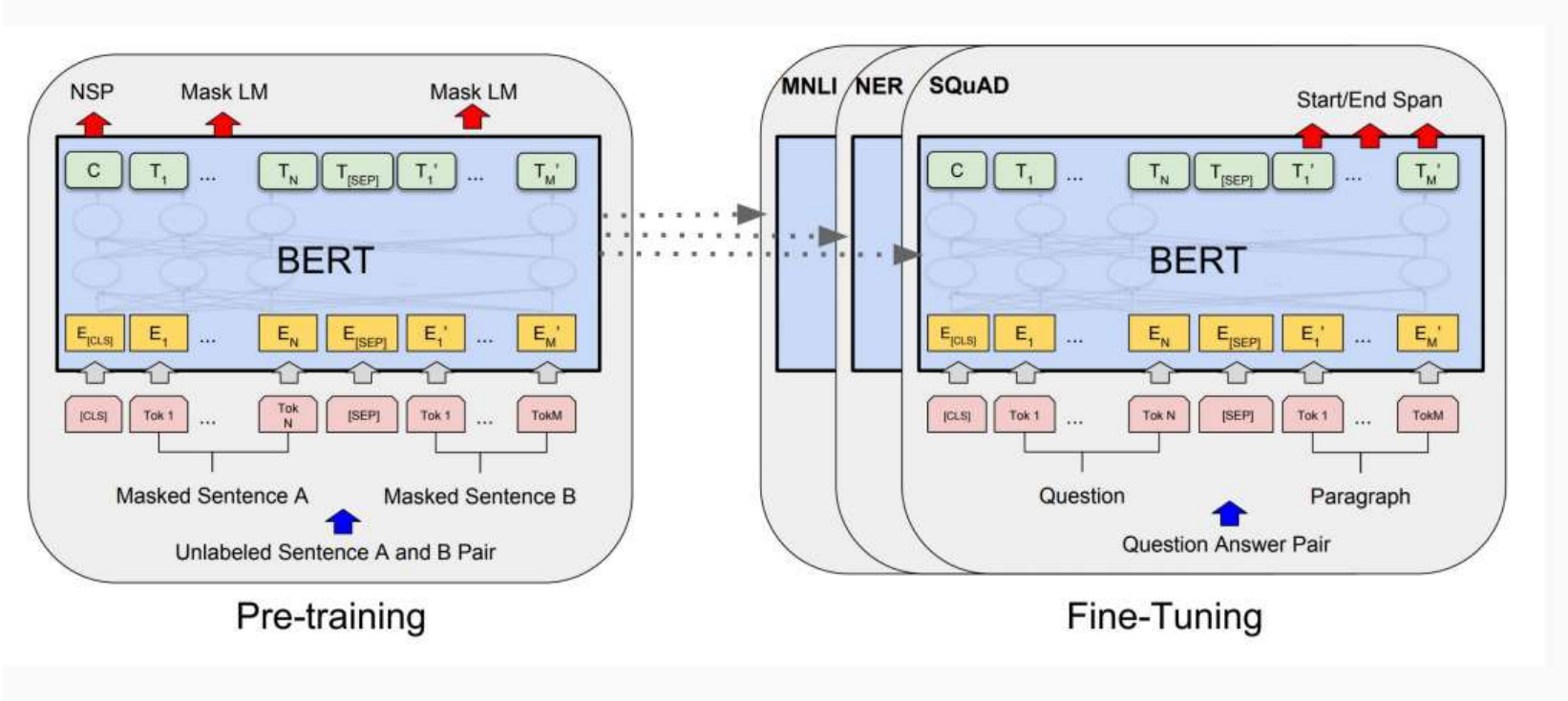
Bert: Pre-training of deep bidirectional transformers for language understanding (*Devlin et al., HLT-NAACL 2018*)



-50% true second sentences
-50% random second sentences

Bert: Pre-training of deep bidirectional transformers for language understanding (*Devlin et al., HLT-NAACL 2018*)

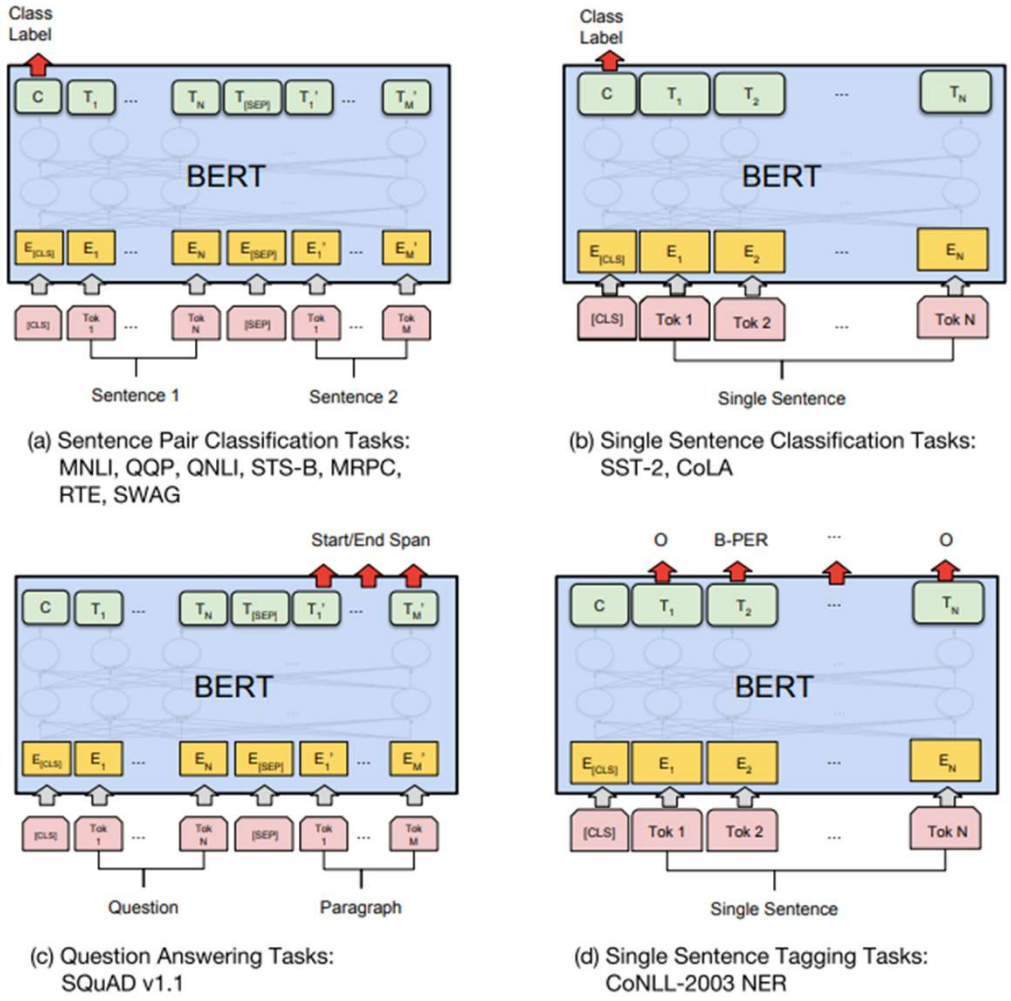
- Pretraining : Masked LM + Next Sentence Prediction
- Finetuning : Other NLP tasks



Bert: Pre-training of deep bidirectional transformers for language understanding (*Devlin et al., HLT-NAACL 2018*)

MNLI (NL inference dataset)
QQP (Quaro Question Pairs)
Semantic equivalence)
QNLI (NL inference dataset)
STS-B (texture similarity)
MRPC (paraphrase, Microsoft)
RTE (textual entailment)
SWAG (commonsense inference)
SST-2 (sentiment)
CoLA (linguistic acceptability)

SST (Stanford sentiment treebank): 215k phrases with fine-grained sentiment labels in the parse trees of 11k sentences.



How to Fine-Tune BERT for Text Classification? (*Sun et al., CCL 2018*)

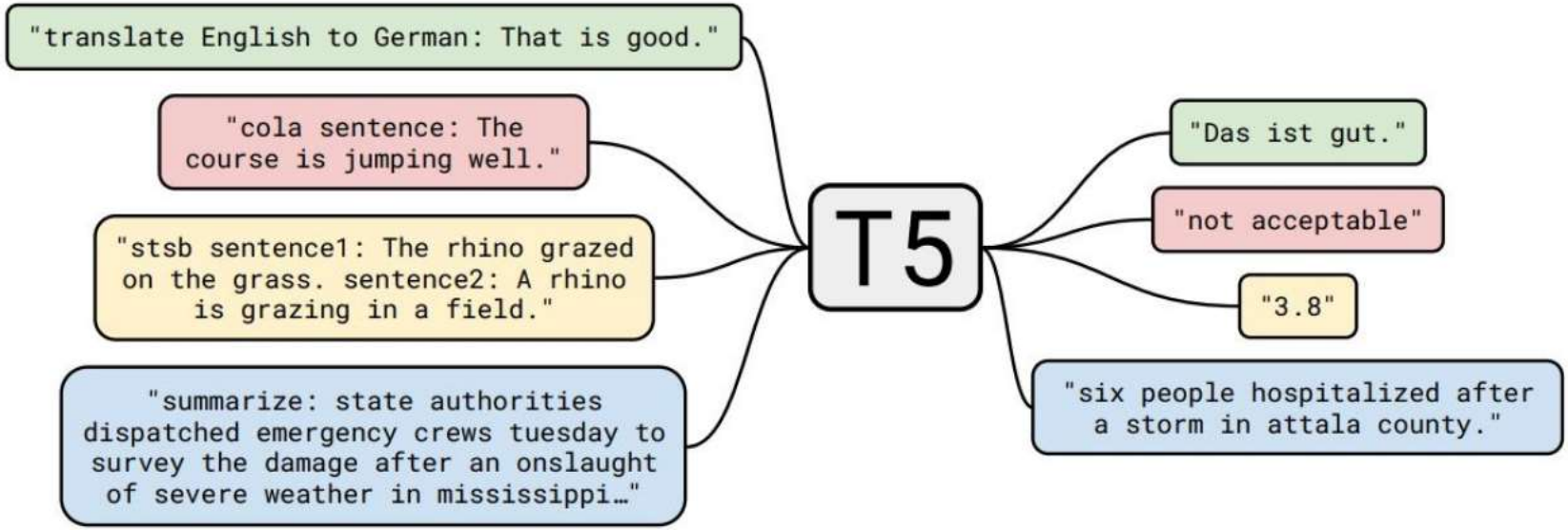
- The top layer of BERT is more useful for text classification
- With an appropriate layer-wise decreasing learning rate, BERT can overcome the catastrophic forgetting problem
- Within-task and in-domain further pre-training can significantly boost its performance
- A preceding multi-task fine-tuning is also helpful to the single-task fine-tuning, but its benefit is smaller than further pre-training
- BERT can improve the task with small-size data

RoBERTa: A Robustly Optimized BERT Pretraining Approach (*Liu et al., ICLR 2019*)

- Improved masking and pre-training data slightly
- Remove the NSP task in BERT
- Trained BERT for more epochs and/or on more data
  - Showed that more epochs alone helps, even on same data
  - More data also helps

Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (*Raffel et al., jmlr 2019*)

- Text-to-Text

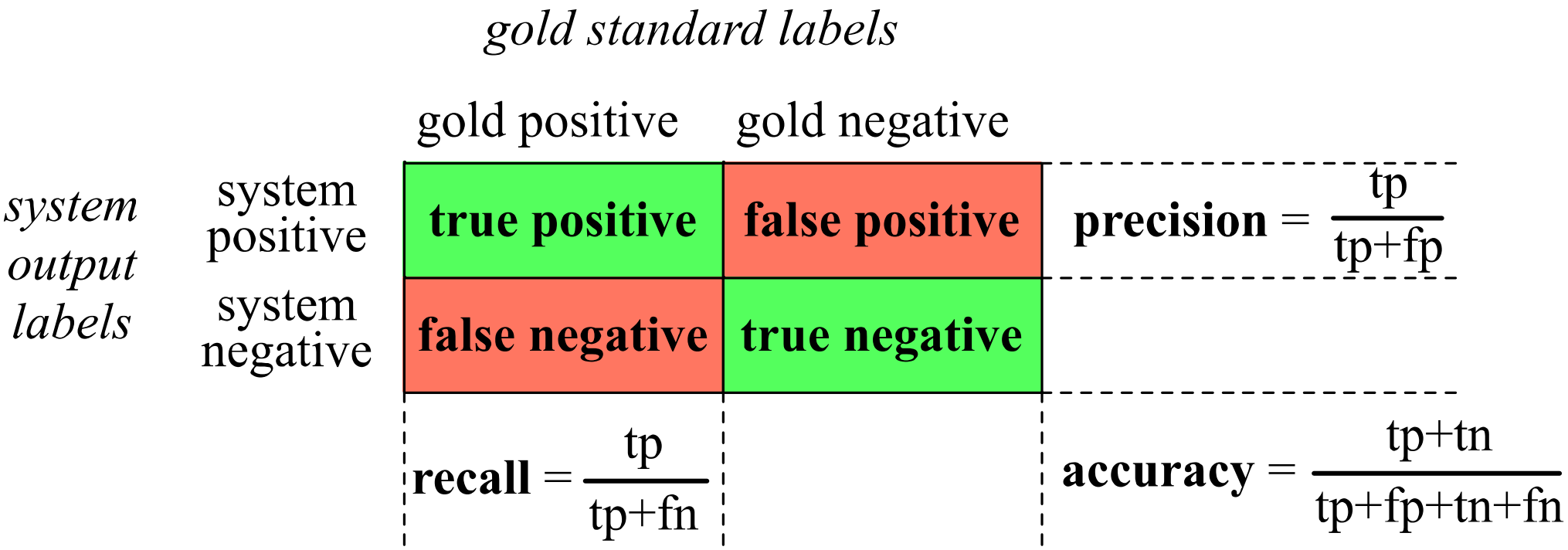


Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (*Raffel et al., jmlr 2019*)

Task Name	Explanation
CoLA	Classify if a sentence is grammatically correct
RTE	Classify whether if a statement can be deducted from a sentence
MNLI	Classify for a hypothesis and premise whether they contradict or contradict each other or neither of both (3 class).
MRPC	Classify whether a pair of sentences is a rephrasing of each other (semantically equivalent)
QNLI	Classify whether the answer to a question can be deducted from an answer candidate.
QQP	Classify whether a pair of questions is a rephrasing of each other (semantically equivalent)
SST2	Classify the sentiment of a sentence as positive or negative
STSB	Classify the sentiment of a sentence on a scale from 1 to 5 (21 Sentiment classes)
CB	Classify for a premise and a hypothesis whether they contradict each other or not (binary).
COPA	Classify for a question, premise, and 2 choices which choice the correct choice is (binary).
MultiRc	Classify for a question, a paragraph of text, and an answer candidate, if the answer is correct (binary)
WiC	Classify for a pair of sentences and a disambiguous word if the word has the same meaning in both sentences.
WSC/DPR	Predict for an ambiguous pronoun in a sentence what it is referring to.
Summarization	Summarize text into a shorter representation.
SQuAD	Answer a question for a given context.
WMT1	Translate English to German
WMT2	Translate English to French
NQ	Closed Book Answering on Natural Questions(nq) Corpus

https://huggingface.co/mrm8488/t5-base-finetuned-imdb-sentiment

Evaluation Metrics (Confusion Matrix)



Evaluation Metrics: Accuracy

- Why don't we use **accuracy** as our metric?
- Imagine we saw 1 million tweets:
  - 100 of them talked about Apple.
  - 999,900 talked about something else
- We could build a dumb classifier that just labels every tweet "not about Apple"
  - It would get 99.99% accuracy!
  - But useless! Doesn't return the comments we are looking for!
  - That's why we use **precision** and **recall** instead

Evaluation Metrics: Precision and Recall

- Our dumb pie-classifier
  - Just label nothing as "about Apple“

Accuracy=99.99%

but Recall = 0

(it doesn't get any of the 100 Apple tweets)

- Precision and recall, unlike accuracy, emphasize true positives:
  - finding the things that we are supposed to be looking for.

Evaluation Metrics: F

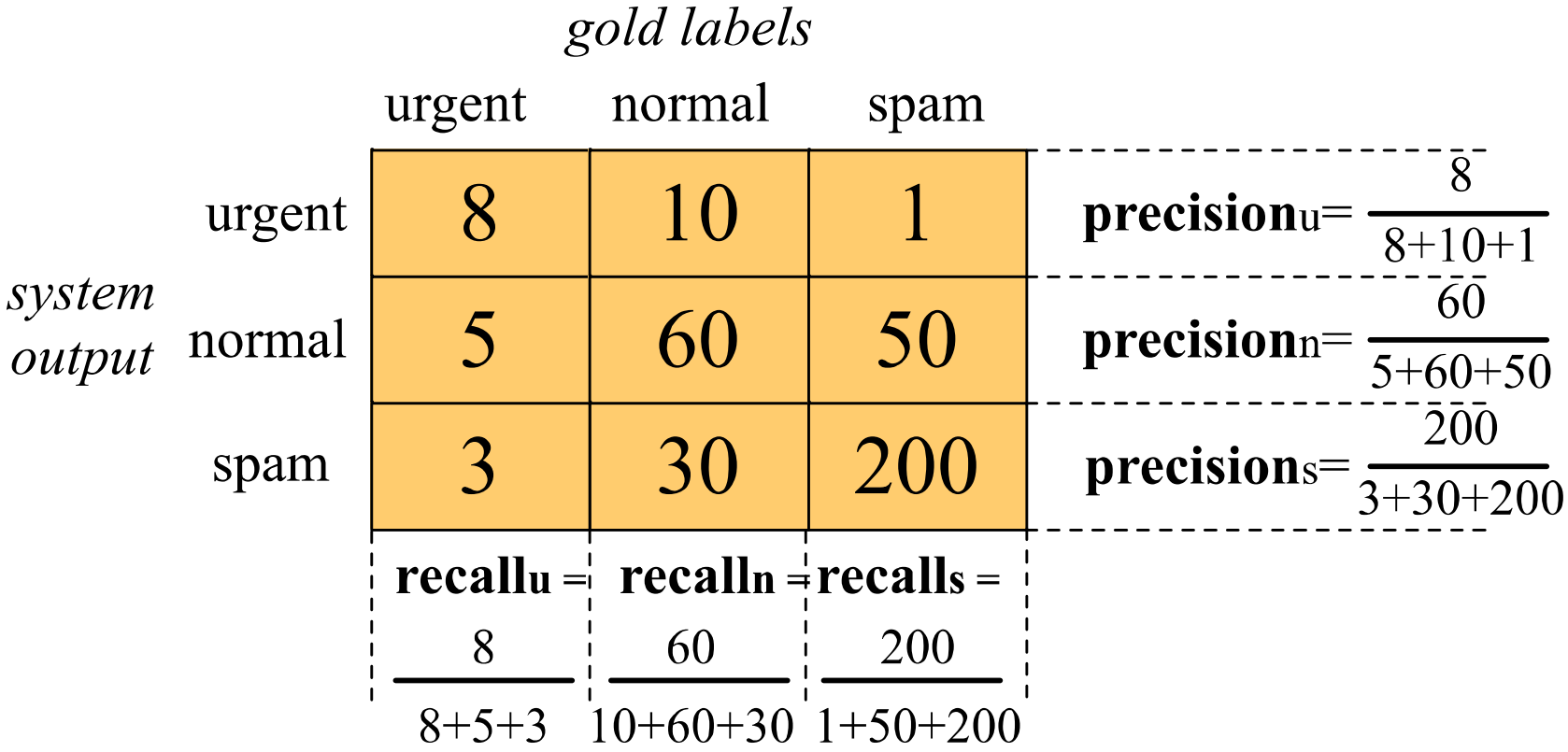
- F measure: a single number that combines P and R:

$$F_\beta = \frac{(\beta^2 + 1)PR}{\beta^2 P + R}$$

- We almost always use balanced F_1 (i.e., $\beta = 1$)

$$\mathrm{F}_1 = \frac{2PR}{P + R}$$

Evaluation Metrics:
Confusion Matrix for 3-class classification



Evaluation Metrics:

- How to combine P/R from 3 classes
- Macro-averaging:
  - compute the performance for each class, and then average over classes
- Micro-averaging:
  - collect decisions for all classes into one confusion matrix
  - compute precision and recall from that table.

Evaluation Metrics: Macro-averaging and Micro-averaging

Class 1: Urgent

	true urgent	true not
system urgent	8	11
system not	8	340

$$\text{precision} = \frac{8}{8+11} = .42$$

Class 2: Normal

	true normal	true not
system normal	60	55
system not	40	212

$$\text{precision} = \frac{60}{60+55} = .52$$

Class 3: Spam

	true spam	true not
system spam	200	33
system not	51	83

$$\text{precision} = \frac{200}{200+33} = .86$$

Pooled

	true yes	true no
system yes	268	99
system no	99	635

$$\text{microaverage precision} = \frac{268}{268+99} = \mathbf{.73}$$

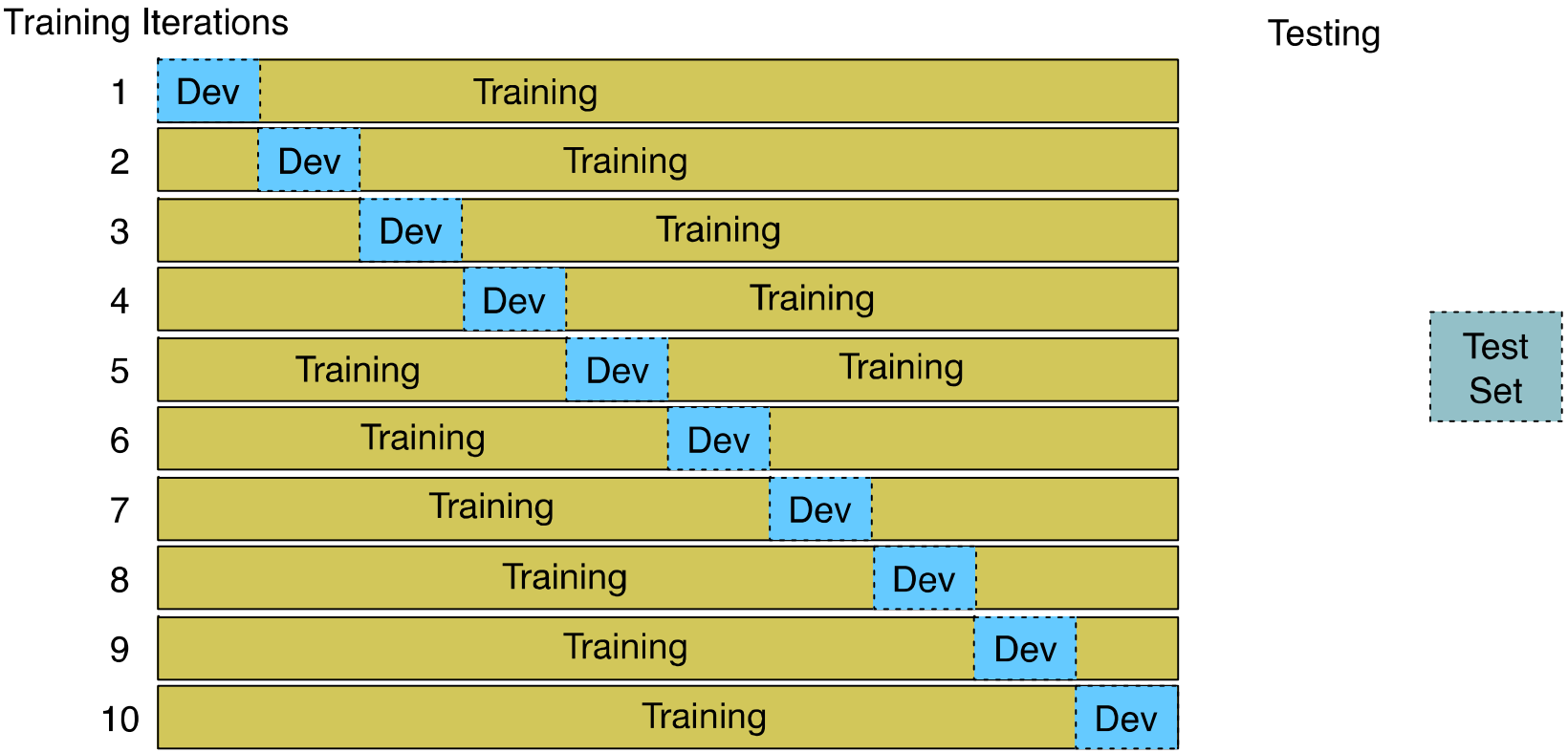
$$\text{macroaverage precision} = \frac{.42+.52+.86}{3} = \mathbf{.60}$$

Evaluation : Development Test Sets

- Train on training set, tune on dev set, report on test set
  - This avoids overfitting (‘tuning to the test set’)
  - More conservative estimate of performance
  - But paradox: want as much data as possible for training, and as much for dev; how to split?

Evaluation : Cross-validation

- Pool results over splits, Compute pooled dev performance



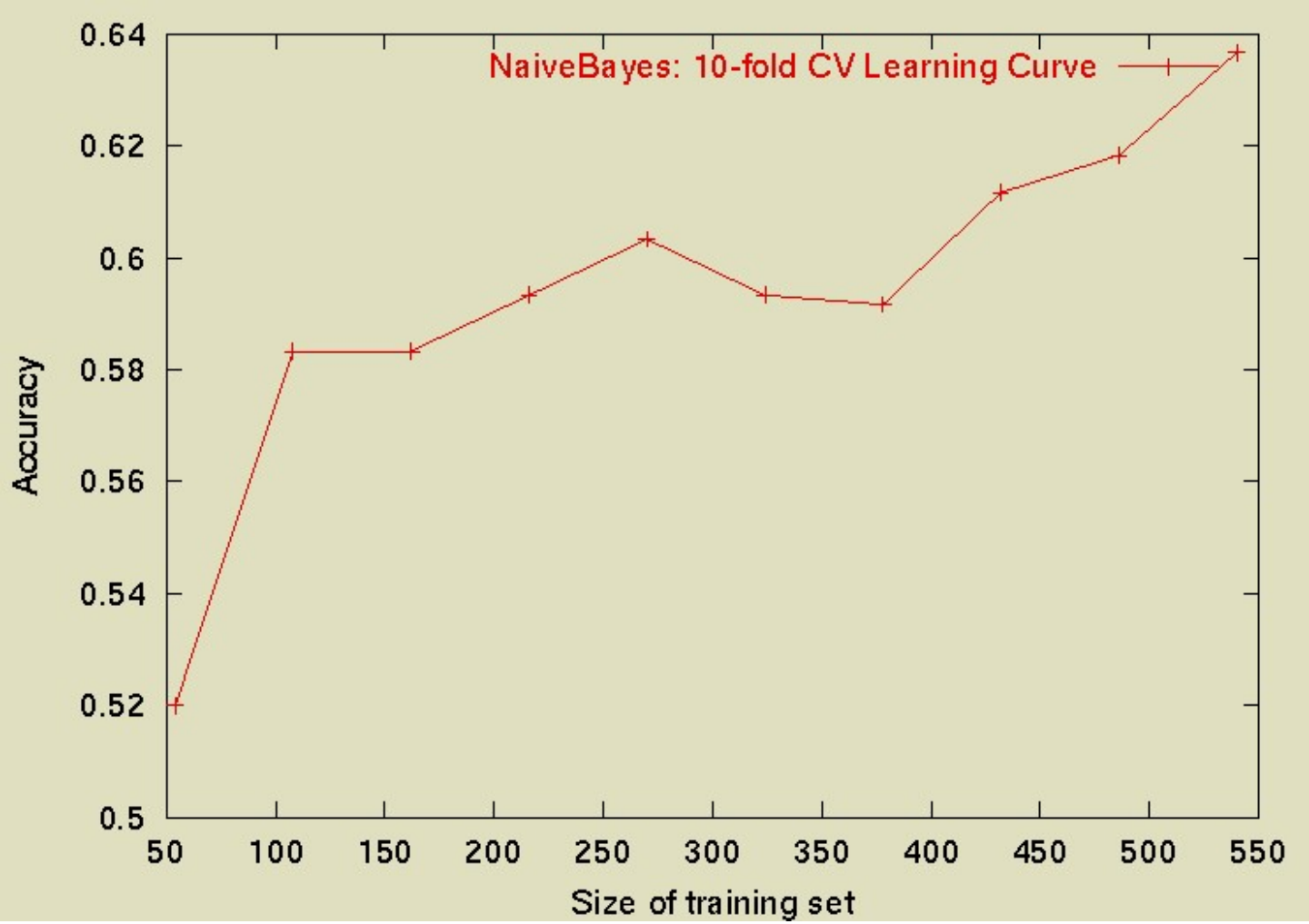
Learning Curves

- In practice, labeled data is usually rare and expensive.
- Would like to know how performance varies with the number of training instances.
- *Learning curves* plot classification accuracy on independent test data (*Y* axis) versus number of training examples (*X* axis).

N-Fold Learning Curves

- Want learning curves averaged over multiple trials.
- Use *N*-fold cross validation to generate *N* full training and test sets.
- For each trial, train on increasing fractions of the training set, measuring accuracy on the test data for each point on the desired learning curve.

Sample Learning Curve



The Next Lecture

- Lecture 11

  Text Clustering